

第 22 章: Modules: The Big Picture (模块: 全景图)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 第五部分(模块与包)的开篇章。前四部分你学会了语句、函数、作用域, 现在要回答"如何把多个文件组织成一个完整程序"。本章站在宏观视角讲解模块的三大角色——代码复用、消除冗余、命名空间隔离, 并完整剖析 import 语句的底层三步流程(查找→编译→运行), 为第 23 章的 import 语句细节和第 24 章的包(package)打好地基。

22.1 开篇引言 (Chapter Introduction)

英文原文

This chapter begins our in-depth look at the Python module—the highest-level program organization unit, which packages program code and data for reuse, and provides self-contained namespaces that minimize variable name clashes across your programs. Modules were introduced in Chapter 3, and we've been using them more and more since Chapter 16, but this p

art of the book provides a focused, detailed look at this Python tool. This first chapter in Part V reviews module basics, and offers a general look at the role of modules in the overall structure of programs. In the chapters that follow, we'll dig into the coding details behind that theory. Along the way, we'll also flesh out module fine points omitted so far—you'll learn about reloads, the name and all attributes, package imports, relative import syntax, namespace packages, the get_attr hook, the main.py file, and so on. Because modules and classes are really just glorified namespaces, this part formalizes namespace concepts as well.

中文翻译

本章开始深入探讨 Python 的模块 (module) ——最高层的程序组织单元，它把程序代码和数据打包起来以供复用，并提供自包含的命名空间 (namespace)，最大限度地减少程序之间的变量名冲突。模块在第 3 章首次引入，自第 16 章以来我们一直在越来越频繁地使用它们，但本书的这一部分将对这个 Python 工具进行一次聚焦而详细的审视。作为第五部分的第一章，本章回顾模块的基础知识，并从一个总体视角看待模块在程序整体结构中所扮演的角色。在接下来的章节中，我们将深入探讨这些理论背后的编码细节。在这一过程中，我们还会补全此前略过的模块细节——你将学到重载 (reload)、name 与 all 属性、包导入 (package imports)、相对导入语法、命名空间包 (namespace packages)、getattr 钩子、main.py 文件等等。因为模块和类 (class) 其实不过是“华丽版”的命名空间，所以这

一部分也把命名空间的概念正式化。

深度理解

·核心概念：模块是 Python 程序的最高级组织单元。想象一栋写字楼——语句是砖，函数是房间，模块就是整层楼，而命名空间是层与层之间“谁也看不见谁”的隔断墙。本章就是这张“楼层平面图”。

·底层实现：模块对象本质上是一个带有 dict 字典的对象，所有顶层名字都被存进这个字典里，作为模块的属性。所谓“模块就是命名空间”，在 CPython 里体现为模块对象 + 属性字典。

·设计原因：为什么把“程序怎么拆文件”单独开一篇（Part V）来讲？因为程序一旦变大，单体文件必然不可维护。模块是 Python 复用代码的顶层手段，先建立整体图景，再学细节，符合“宏观→微观”的学习路径。

·解决什么实际问题：变量名冲突（不同文件各用各的 x 互不干扰）、代码复用（写一次，处处导入）、程序可维护性（功能按文件切分）。

·初学者误区：以为模块就是“一个 .py 文件”这么简单。实际上模块可以是 C 扩展、ZIP 里的文件、目录（包），甚至内存镜像——文件只是最常见的形式。另外，“命名空间”不是玄学，就是“一个字典里装着一堆名字”。

22.2 Module Essentials（模块要点）

英文原文

In simple and concrete terms, modules typically correspond to Python source code files. Each file of code is a module automatically, and modules import other modules to use the names they define. Modules might also correspond to extensions coded in external languages such as C, Java, or C#, and even to entire directories in package imports, which extend the model for nested files. In all their forms, modules are processed with two statements and one tool: `import` — Lets a client (importer) fetch a module as a whole from — Allows clients to fetch particular names from a module `importlib.reload` — Provides a way to reload a module's code without stopping Python We've used imports in prior examples to load both Python standard-library modules, as well as code files that reside in the current directory—the one we are in when launching the REPL or a script file. While straightforward on the surface, these tools imply an underlying model that's richer than you might think. Before we delve into its details, though, let's begin by getting a handle on the purpose of modules in our Python programs.

中文翻译

用简单具体的话说，模块通常对应 Python 源代码文件。每个代码文件自动就是一个模块，模块之间通过导入（`import`）彼此，以使用对方定义的名字。模块也可能对应以 C、J

ava、C# 等外部语言编写的扩展 (extension) , 甚至对应包导入 (package import) 中的整个目录——这扩展了模型, 使其支持嵌套文件。无论何种形式, 模块都由两个语句和一件工具来处理: import (导入) ——让客户端 (导入方) 把整个模块作为一个整体取走 from (从...导入) ——允许客户端从模块中取出特定名字 importlib.reload (重载) ——提供一种不必停止 Python 就能重新加载模块代码的方式我们在前面的例子中已经用导入加载过 Python 标准库模块, 以及位于当前目录的代码文件——即我们启动 REPL 或脚本文件时所在的那个目录。虽然表面上很简单, 但这些工具背后的底层模型比你想的要丰富得多。在深入细节之前, 让我们先弄清楚模块在我们 Python 程序中的用途。

深度理解

·核心概念: 处理模块的核心工具只有三样: import (取整个模块)、from (取模块里的特定名字)、importlib.reload (不重启 Python 重新执行模块代码)。本书后续所有关于模块的内容都围绕这三者展开。

·底层实现: import b 在字节码层面是一条 IMPORTNAME 指令; from b import x 在 IMPORTNAME 之后还会执行 IMPORTFROM 指令。importlib.reload 则是重新运行模块文件的字节码并更新 sys.modules 中的模块对象内容。

·设计原因: 只给两个语句一个工具, 是因为模块的绝大部分需求就这两种: 要整包, 还是要单件。这种"最小接口"设计让导入系统简单易学, 复杂的文件解析全部隐藏在语句背后。

·解决什么实际问题：加载标准库（`import sys`）、加载自己写的代码文件、以及运行时更新模块代码（`reload` 用于动态定制，如热更新配置）。

·初学者误区：把 `import` 理解为“把另一文件的文本粘贴进来”，或以为 `reload` 是“再 `import` 一次”。前者是 C 语言 `#include` 的思维方式，后者会惊讶地发现“重复 `import` 根本不重新执行代码”——这正是 `sys.modules` 缓存导致的（见 22.6 节）。

22.3 Why Use Modules? (为什么使用模块?)

英文原文

In short, modules provide an easy way to organize components into a system, by serving as self-contained packages of variables known as namespaces. All the names defined at the top level of a module file become attributes of the imported module object, but a file's names can't be seen without imports, and don't clash with names in other files. This model is related to the scopes we studied in the last part of this book. As we learned in Chapter 17, imports give access to names in a module's global scope. The file surrounding a function is always that function's own global scope, but imports allow one file to see the global names of another file as attributes. That is, the module file's

global scope morphs into the module object's attribute namespace when it is imported. Ultimately, this allows us to link individual files into a larger program system, with three primary benefits: Code reuse — Modules make code permanent. Because a module's code is saved in a file, you can both run it multiple times and use it in multiple programs. As you've learned, code you type at a Python REPL goes away when you exit Python, but code in module files can be rerun as many times as you wish. Moreover, the tools you define in modules may be reused by any number of external clients, and in programs coded both now and in the future. Minimizing redundancy — Module reuse naturally enables shared copies of common code. If more than one file uses the same or similar code, you can write it once in a module that can then be imported by many clients. For example, in the prior chapter's benchmarking examples, test scripts imported common timer and test-runner functions, instead of repeating them. In other words, modules—like functions—help us factor code to avoid the redundancy that results from copy-and-paste programming. As for functions, this minimizes work when common code must be changed. Namespace partitioning — Modules are also the highest-level namespace structure in a Python program. Although they are fundamentally just packages of names, these packages are also self-contained—you can never see a name in another file unless you

u explicitly import that file. Much like the local scope of functions, this helps avoid name clashes across your programs. In fact, you can't avoid this feature—everything "lives" in a module. Because both the code you run and the objects you create are always implicitly enclosed in modules, modules group components by nature. At least that's the abstract story. To truly understand the role of modules in a Python system, we need to digress for a moment and explore the general structure of a Python program.

中文翻译

简而言之，模块提供了一种把组件组织成系统的简便方式：它作为自包含的变量包（称为命名空间）来服务。模块文件顶层定义的所有名字都会成为被导入模块对象的属性（attribute），但一个文件里的名字，未经导入就不可见，也不会与其他文件中的名字冲突。这个模型与我们上一部分研究的作用域（scope）密切相关。如第 17 章所学，导入让我们能够访问模块全局作用域中的名字。包围函数的那个文件永远是函数自身的全局作用域，但导入允许一个文件把另一个文件的全局名字当作属性来看待。也就是说，模块文件的全局作用域在被导入时“变身”（morph）为模块对象的属性命名空间。归根结底，这让我们能够把一个个独立文件链接成更大的程序系统，带来三大主要好处：代码复用（Code reuse）——模块让代码“永生”。因为模块代码保存在文件中，你既可以多次运行它，也可以把它用在多个程序中。如你所知，在 Python REPL 里敲的代码在你退出 Python 时就消失了，但模块文件里的代码可以想运行多少次就运行多

少次。此外，你在模块中定义的工具可以被任意多的外部客户端复用，也可以用在你现在和将来编写的程序里。最小化冗余（Minimizing redundancy）——模块复用自然带来了公共代码的共享副本。如果不止一个文件使用相同或相似的代码，你可以在一个模块里写一次，再被许多客户端导入。例如，上一章的基准测试（benchmarking）例子中，测试脚本导入了公共的计时器和测试运行器函数，而不是重复编写它们。换句话说，模块——和函数一样——帮助我们分解代码，避免“复制粘贴式编程”造成的冗余。和函数一样，当公共代码需要修改时，这也把工作量降到了最低。命名空间划分（Namespace partitioning）——模块还是 Python 程序中最高层的命名空间结构。虽然它们本质上只是名字的包，但这些包也是自包含的——除非你显式导入某个文件，否则你永远看不到另一个文件里的名字。这与函数的局部作用域非常相似，有助于避免程序间的名字冲突。事实上，你根本躲不开这个特性——一切都“活”在某个模块里。因为你运行的代码和你创建的对象总是被隐式地包裹在模块中，模块天然地对组件进行了分组。至少这是抽象层面的说法。要真正理解模块在 Python 系统中的作用，我们需要暂时离题，探究一下 Python 程序的总体结构。

深度理解

- 核心概念：用模块的三大好处——代码复用、消除冗余、命名空间划分——解释“为什么要用模块”。其中命名空间划分是最本质的一条：模块的全局作用域在导入时“变身”为模块对象的属性命名空间。
- 底层实现：“变身”的过程在 CPython 里真实存在：import b 会创建 ModuleType 对象，然后逐条执行 b.py 的字

节码，顶层每条赋值语句都会执行 STOREATTR，把名字存进模块对象的 dict。全局作用域和模块属性字典，本质是同一个字典的两种视角。

·设计原因：为什么 Python 不像 C 那样"文本包含"（#include）而是"对象属性"？因为文本包含会造成同名覆盖和编译依赖混乱，而运行时属性访问让"谁导入谁"完全显式、可按需动态进行。

·解决什么实际问题：三个场景——同一份代码想在多个程序里用（复用）、同一段逻辑被多处复制导致改一处漏三处（冗余）、两个模块都定义了 max 却互不干扰（冲突）。

·初学者误区：以为"所有文件共享一套全局名字"（那是 C 预处理器的世界观）；或者反过来，以为模块命名空间必须"手动创建"。实际上一切都是自动的：任何 .py 文件天然就是模块，顶层名字天然就是属性。

22.4 Python Program Architecture (Python 程序架构)

英文原文

So far in this book, many examples have sugarcoated the complexity of Python programs. In practice, programs usually involve more than just one file. For all but the simplest scripts, your programs will take the form of multifile systems—as the code-benchmarking

programs of the preceding chapter illustrated. Even if you can get by with coding a single file yourself, you will almost certainly wind up using external files that someone else has already written. This section reviews the general architecture of Python programs—the way you divide a program into a collection of source code files (a.k.a. modules) and link the parts into a whole. As you'll see, Python fosters a modular program structure that groups functionality into coherent and reusable units, in ways that are almost automatic. Along the way, this section also explores the central concepts of Python modules, imports, and object attributes.

中文翻译

到目前为止，本书中的许多例子都把 Python 程序的复杂性“糖衣化”了。实际上，程序通常不止一个文件。除了最简单的脚本之外，你的程序都会采用多文件（multifile）系统的形式——上一章的代码基准测试程序就说明了这一点。即使你自己只写一个文件就能搞定，你几乎也必然会用到别人已经写好的外部文件。本节回顾 Python 程序的总体架构——也就是你如何把程序划分成一组源代码文件（也就是模块），并把各个部分链接成一个整体。你将会看到，Python 培育了一种模块化的程序结构，以几乎自动化的方式把功能组织成内聚、可复用的单元。与此同时，本节也会探讨 Python 模块、导入和对象属性的核心概念。

深度理解

- 核心概念：现实世界的 Python 程序是多文件系统：一个顶层脚本（script）+ 零个或多个模块库。架构 = 怎么拆 + 怎么连。
- 底层实现："连"靠的是 import 语句在运行时把模块对象绑定到变量上；"拆"没有任何强制规定，纯靠约定和组织。
- 设计原因：单文件脚本能跑，但无法分工、无法复用。多文件是规模化的必然。Python 之所以让"文件即模块"自动成立，是为了让拆分几乎零成本——你不需要像 Java 那样声明包名才能拆文件。
- 解决什么实际问题：一个项目多人协作时怎么切分代码、代码怎么跨文件引用、程序入口和工具库怎么区分。
- 初学者误区：分不清"脚本文件"和"模块文件"的职责——脚本负责"做事情"（顶层代码会立即执行），模块负责"定义工具"（顶层代码只是把名字准备好）。同一个文件在一种场景是脚本，在另一种场景可能被当作模块导入。

22.4.1 How to Structure a Program (如何组织一个程序)

英文原文

At a base level, a Python program consists of text files containing Python statements, with one main top-l

level file, and zero or more supplemental files known as modules. Here's how this works. The top-level (a.k.a. script) file contains the main flow of control of your program—this is the file you run to launch your application. The module files are libraries of tools, used to group components used by the top-level file, and possibly elsewhere. Top-level files use tools defined in module files, and modules use tools defined in other modules. Although they are files of code too, module files generally don't do anything when run directly; rather, they define tools intended for use in other files. A file imports a module to gain access to the tools it defines, which are known as its attributes—variable names attached to objects such as functions. Ultimately, we import modules and access their attributes to use their tools.

中文翻译

在最基本的层面上，一个 Python 程序由包含 Python 语句的文本文件组成：一个主要的顶层（top-level）文件，外加零个或多个称为模块的补充文件。它是这样运作的：顶层文件（又称脚本，script）包含程序的主控制流——这是你运行以启动应用程序的文件。模块文件是工具库，用来组织顶层文件以及其他地方要用的组件。顶层文件使用模块文件中定义的工具，模块也使用其他模块中定义的工具。虽然模块文件本身也是代码文件，但它们通常不会在直接运行时“做什么事”；相反，它们定义供其他文件使用的工具。一个文件导入某个模块，是为了访问它定义的工具——这些工

具被称为模块的属性 (attribute) , 即挂在函数等对象上的变量名。归根结底, 我们导入模块并访问其属性, 来使用它们的工具。

深度理解

·核心概念: 程序结构 = 一个"总指挥" (顶层脚本, 跑起来的文件) + 若干"工具库" (模块)。工具用属性 (attribute) 的形式向外暴露。

·底层实现: 顶层脚本由 `python a.py` 命令以 main 模块的身份运行; 被导入的模块则按名字创建各自独立的模块对象。两者执行的字节码完全相同, 差别只在于"谁被当作主程序"。

·设计原因: 把"控制流"和"工具定义"分开, 是程序架构中最古老也最有效的组织原则。脚本负责流程编排 (像导演), 模块负责功能实现 (像演员)。

·解决什么实际问题: 入口文件该放什么、工具文件该放什么; 如何让多个脚本共享同一套底层实现。

·初学者误区: 直接运行模块文件时"什么都没发生"就以为代码写错了——其实模块被直接运行时, 顶层 `def` 只是定义了函数, 没有任何调用, 所以看起来"没动静", 这是正常的设计意图。

22.4.2 Imports and Attributes (导入与属性)

英文原文

To make this a bit more concrete, Figure 22-1 sketches the structure of a Python program composed of three files: `a.py`, `b.py`, and `c.py`. The file `a.py` is chosen to be the top-level script file; it will be a simple text file of statements, which is executed from top to bottom when launched. The files `b.py` and `c.py` are modules; they are simple text files of statements as well, but they are not usually launched directly. Instead, as explained previously, modules are normally imported by other files that wish to use the tools the modules define. Figure 22-1. Program architecture in Python For instance, suppose the file `b.py` in Figure 22-1 defines a function called `job` for external use, per Example 22-1 (and ignoring `c.py` for the moment). As you learned when studying functions in Part IV, `b.py` will contain a Python `def` statement to generate the function, which you can later run by passing values in parentheses after the function's name. Example 22-1. `b.py` (module)

```
python def job(tool): print(tool,'coder')
```

This function probably seems trivial at this point in this book, but we're keeping it simple to focus on module basics. Now, suppose `a.py` wants to use `job`. To this end, it might contain Python statements like those in Example 22-2. Example 22-2. `a.py` (script)

```
python import b b.job('Python')
```

The first of these, a Python import statement, gives the file `a.py` access to everything defined by top-level code—that is, code not nested inside a function or class—within the file `b.py`. The code `import b` roughly means: Load the file `b.py` (unless it's already loaded), and give me access to all its attributes through the name `b`. To satisfy such goals, import (and, as you'll see later, from) statements execute and load other files on request. More formally, in Python, cross-file module linking is not resolved until such import statements are executed at runtime; their net effect is to assign module names—simple variables like `b`—to loaded module objects. In fact, the module name used in an import statement serves two purposes: it identifies the external file to be loaded (by base name `b` here), but it also becomes a variable assigned to the loaded module. Similarly, objects defined by a module's code are also created at runtime, while the import is executing: import literally runs statements in the target file one at a time to create its contents. Along the way, every name assigned at the top level of the file becomes an attribute of the module, accessible to importers. For example, the second of the statements in `a.py` calls the function `job` defined in the module `b`—and created by running its `def` statement during the import—using object attribute notation. The code `b.job` means: Fetch the value of the name `job` that lives within

the object `b`. This happens to be a callable function in our example, so we pass a string in parentheses ('Python'). If you run `a.py`, the words "Python coder" will be printed—hardly a shocker if you've read prior chapters, but illustrative. As we've seen, the `object.attribute` notation is general and pervasive in Python code because most objects have useful attributes that are fetched with the `"."` operator. Some attributes reference callable objects like functions that take action (e.g., a salary computer), while others are simple values that denote data (e.g., a person's name). Imports are similarly general because any file can import tools from any other file. For instance, the file `a.py` in Figure 22-1 may import `b.py` to call its function, but `b.py` might also import `c.py` to leverage different tools defined there. In fact, import chains can go as deep as you like: in this example, module `a` can import `b`, which can import `c`, which can import `b` again, and so on. More realistically, in the benchmarking code of the prior chapter, test scripts imported runner modules, which imported timer modules. The main point behind all this is that modules (and module packages, described in Chapter 24) are the topmost level of code reuse in Python. Components coded in module files can be used both in your original program and in any other programs you may write later. For instance, if we later discover that the function `b.job` in Example 22-1 is widely useful, we can deploy it in completely different

programs; all we have to do is import b again from the other programs. While unlikely in this simple demo, modules by nature create packages of reusable tools.

中文翻译

为了让这一点更具体，图 22-1 勾画了一个由三个文件组成的 Python 程序结构：a.py、b.py 和 c.py。a.py 被选为顶层脚本文件；它就是一个简单的语句文本文件，启动时从上到下执行。b.py 和 c.py 是模块；它们同样是简单的语句文本文件，但通常不会直接启动。相反，如前所述，模块通常由其他想要使用其工具的文件来导入。图 22-1. Python 中的程序架构例如，假设图 22-1 中的 b.py 按照示例 22-1 定义了一个供外部使用的函数 job（暂时忽略 c.py）。正如你在第四部分学习函数时所知，b.py 将包含一条 Python `def` 语句来生成这个函数，之后你可以通过在函数名后的括号里传入值来运行它。示例 22-1. b.py（模块）

```
python def job(tool): print(tool,'coder')
```

这个函数在本书的这个阶段看来可能微不足道，但我们故意保持简单，以便聚焦模块基础。现在，假设 a.py 想使用 job。为此，它可以包含类似示例 22-2 的 Python 语句。示例 22-2. a.py（脚本）

```
python import b b.job('Python')
```

其中第一条是 Python `import` 语句，它让 a.py 能够访问 b.py 文件中顶层代码（即不在函数或类内部嵌套的代码）定义的一切。`import b` 这条代码大致意思是：加载文件 b.py（如果还没加载的话），并让我通过名字 b 访问它的所有属性。为了满足这类目标，`import`（以及后面你会看到的 `from`）语句会按要求执行并加载其他文件。更正式地说，在 Python 中，跨文件的模块链接直到这些 `import` 语句在运行时被执行为止才得

以解决；它们的净效果是把模块名——像 `b` 这样的简单变量——赋值给已加载的模块对象。事实上，`import` 语句中使用的模块名有两个用途：它标识了要加载的外部文件（这里的基本名是 `b`），同时也成为赋值给已加载模块的一个变量。同样，模块代码定义的对象也是在运行时、导入执行期间创建的：`import` 字面上逐条运行目标文件中的语句来创建其内容。在这一过程中，文件顶层被赋值的每个名字都会成为模块的一个属性，供导入方访问。例如，`a.py` 中的第二条语句使用对象属性表示法调用函数 `job`——这个函数定义在模块 `b` 中，是在导入期间运行其 `def` 语句时创建的。`b.job` 这段代码的意思是：取出对象 `b` 内部名为 `job` 的名字所对应的值。在我们的例子中它恰好是一个可调用函数，所以我们在括号里传入了字符串（`'Python'`）。如果你运行 `a.py`，会打印出 `"Python coder"`——如果你读过前面的章节，这并不意外，但它很有说明性。正如我们所见，对象.属性（`object.attribute`）这种表示法在 Python 代码中既普遍又通用，因为大多数对象都有用属性，可以用 `"."` 操作符取出。有些属性引用的是可调用对象，比如采取行动的函数（例如一个“薪资计算器”），另一些则是表示数据的简单值（例如一个人的名字）。导入同样具有通用性，因为任何文件都可以从任何其他文件导入工具。例如，图 22-1 中的 `a.py` 可以导入 `b.py` 来调用它的函数，但 `b.py` 也可能导入 `c.py` 来使用那里定义的不同工具。事实上，导入链可以无限深：在这个例子中，模块 `a` 可以导入 `b`，`b` 可以导入 `c`，`c` 又可以导入 `b`，如此等等。更现实地说，在上一章的基准测试代码中，测试脚本导入了 `runner` 模块，`runner` 模块又导入了 `timer` 模块。这一切背后的要点是：模块（以及第 24 章描述的模块包）是 Python 中最高层次的代码复用。模

块文件中的组件既可以用在你原本的程序中，也可以在你以后写的任何其他程序中。例如，如果我们后来发现示例 2-2-1 中的 `b.job` 函数用途广泛，我们可以把它部署到完全不同的程序中；我们要做的只是从其他程序中再次导入 `b`。虽然在这个简单的演示中不太可能，但模块天生就创造可复用工具包。

深度理解

- 核心概念：顶层脚本 + 模块 + 导入 + 属性，四者构成 Python 程序架构的全部要素。`import b` 的名字 `b` 一石二鸟：既是文件名，又是变量名。

- 底层实现：执行 `import b` 时，CPython 完成三步（查找→编译→运行）后，把模块对象绑定到局部命名空间的 `b` 上（执行 `STORENAME` 指令）。`b.job` 是属性访问：先取出 `b` 模块对象，再到其 `dict` 里查 `job`（`LOADATTR` 指令）。运行 `a.py` 时，顶层 `import` 和 `b.job('Python')` 都是立即执行的语句。

- 设计原因：把“文件定位”和“名字绑定”合成一个语句，省去了 C 语言里“头文件 + 链接器”两套机制。对象属性表示法让跨文件访问与对象内部访问语法完全统一——`b.job` 和 `person.name` 没有任何机制上的区别。

- 解决什么实际问题：程序入口文件想用工具库里的函数；一个工具模块想调用另一个工具模块（导入链）；写完的工具想被未来的其他项目复用。

- 初学者误区：以为 `import b` 之后文件里所有名字都“自动可见”——错，必须通过 `b.xxx` 取；以为 `import` 是“把

文件内容粘进来"——错，它只创建对象并绑定变量；以为函数被"搬运"进了 a.py——错，函数对象活在 b 模块里，a.py 只是拿到了引用。

代码分析

```
# b.py
def job(tool):
    print(tool, 'coder')    # + 'coder'

# a.py
import b                    # b.py  b
b.job('Python')            # b  job  'Python'
```

解释器如何处理：

1. 运行 a.py 时，解释器先把它编译成字节码，然后逐条执行顶层语句。
2. 执行到 import b：解释器检查 sys.modules 中是否有 b，没有则走三步流程——在 sys.path 里找到 b.py、编译成字节码、执行其代码。b.py 里的 def job(...) 被执行，创建函数对象，并作为属性 job 存进模块对象 b 的 dict 中。最后把模块对象赋给 a.py 顶层命名空间里的名字 b。
3. 执行到 b.job('Python')：先取模块对象 b，再取它的 job 属性（一个函数对象），然后传入字符串'Python'调用它。
4. 函数体执行 print('Python','coder')，终端输出 Python coder。

设计思想：import 的名字承担"文件标识 + 变量绑定"双

重职责，既免去了写文件路径的麻烦，又让导入方拥有一个显式的前缀（b.），天然形成命名空间隔离。

22.5 Standard-Library Modules (标准库模块)

英文原文

Notice the rightmost portion of Figure 22-1. As we've seen along the way, some of the modules that your programs will import are provided by Python itself and are not files you will code. Python automatically comes with a large collection of utility modules known as the standard library. This collection, hundreds of modules large at last count, contains platform-independent support for common programming tasks: operating system interfaces, object persistence, text pattern matching, network and internet scripting, GUI construction, multithreading, and much more. None of these tools are part of the Python language itself, but you can use them by simply importing the appropriate modules on any standard Python installation. Because they are standard-library modules, you can also be reasonably sure that they will be available and will work portably on most platforms on which you will run Python code. This book's examples employ a few of the standard library's modules—time, timeit, sys, and os in the last chapter's code, for instance—but we'll r

really only scratch the surface of the library's story here. For a complete look, browse the Python standard-library reference manual, available online at python.org and elsewhere. See Chapter 15 for more on these manuals; the PyDoc tool discussed there also provides library-module info and lists every importable module, including the standard library. Because there are so many standard-library modules, browsing is the best way to get a feel for what tools are available. You can also find tutorials on Python library tools in books that cover application-level programming, but the standard manuals are free, viewable in any web browser, and updated each time Python is rereleased. As of Python 3.10, `sys.stdlibmodule_names` also provides a simple list of all standard-library modules—importable or not:

```
$ python3
>>> len(sys.stdlib_module_names)    #
300
```

中文翻译

请注意图 22-1 最右边的部分。正如我们一路走来所看到的，你的程序将要导入的某些模块是由 Python 自身提供的，而不是你编写的文件。Python 自动附带了一大组实用模块，统称为标准库（standard library）。这个集合——上次清点时已有数百个模块——为常见的编程任务提供了与平台无关的支持：操作系统接口、对象持久化（object persistence）、文本模式匹配、网络与互联网脚本、GUI 构建

、多线程 (multithreading) , 以及更多。这些工具都不是 Python 语言本身的一部分, 但在任何标准 Python 安装上, 你只需导入相应的模块即可使用它们。因为它们是标准库模块, 你还可以比较确信: 在你运行 Python 代码的大多数平台上, 它们都可用, 并且能可移植地工作。本书的例子用到了标准库中的少数模块——例如上一章代码中的 `time`、`timeit`、`sys` 和 `os`——但在这里我们真的只是触及了标准库的皮毛。要看全貌, 请浏览 Python 标准库参考手册, 它可以在 python.org 等网站上在线获取。关于这些手册的更多信息见第 15 章; 那里讨论的 PyDoc 工具也能提供库模块信息, 并列出每一个可导入的模块, 包括标准库在内。因为标准库模块实在太多, 浏览是感受“有哪些工具可用”的最佳方式。你也可以在介绍应用级编程的书里找到 Python 库工具的教程, 但标准手册是免费的、可用任何网页浏览器查看、并且每次 Python 重新发布时都会更新。从 Python 3.10 起, `sys.stdlibmodule_names` 还提供了一个简单的标准库模块列表——不论是否可以导入:

```
$ python3
>>> len(sys.stdlib_module_names)    #
300
```

深度理解

·核心概念: 标准库是 Python 自带的一整套“免费工具”, 它与语言核心分开, 但随安装附带——图 22-1 最右侧那部分画的就是它们。本书开头提到的“电池自带” (batteries included) 口号在这里落地。

·底层实现: 标准库模块分两类——纯 Python 编写的 (大部分) 和 C 语言编写的 (如 `time`、`sys` 的部分功能) 。

它们的目录在安装时被自动写入 `sys.path` (见 22.13 节)。`sys.stdlibmodule_names` 是 Python 3.10 新增的属性，返回一个不可修改的集合 (frozenset)，内容包含子模块和平台相关模块的名字。

·设计原因：把常用功能随语言一起发布，用户就不必到处下载第三方库，也避免了“同一个库 N 种版本各不兼容”的混乱；由官方团队统一维护还能保证跨平台行为一致。

·解决什么实际问题：任何平台上想干“操作系统接口、网络、GUI、多线程”之类的常见活儿，`import` 一行就搞定；可移植性由 CPython 团队替你保证。

·初学者误区：以为标准库是“语言的一部分”（其实不 `import` 就不会被加载）；以为只能上网搜第三方库（先查标准库往往就有现成方案）；以为 `sys.stdlibmodule_names` 里的每个名字都能直接 `import`（部分名字是顶层目录名，不可直接导入）。

代码分析

```
>>> import sys                                # sys
>>> len(sys.stdlib_module_names) #
300
```

解释器如何处理：`sys` 属于“内建 (built-in) + 冻结 (frozen)”模块，解释器启动时就已经驻留内存，所以不需要走三步导入流程。`len()` 作用于 `frozenset` 返回元素个数——依照本书写作时的版本统计为 300 个（含不可直接导入的名字）。这个数字随 Python 版本变化，不必死记。

设计思想：把“库里有什么”做成一个可查询的 API 而不是文档里的承诺，让程序能否静态检查自身依赖的同时，也侧面说明了标准库有多么“厚”。

22.6 How Imports Work (import 是如何工作的)

英文原文

The prior section talked about importing modules without fully explaining what happens when you do so. Because imports are at the heart of program structure in Python, this section goes into more formal detail on the import operation to make this process less abstract. Some C programmers like to compare the Python module import operation to a C `#include`, but they really shouldn't—in Python, imports are not textual insertions of one file into another. They are really runtime operations that perform three distinct steps the first time a program imports a given file: 1. Find the module's file. 2. Compile it to bytecode (if needed). 3. Run the module's code to build the objects it defines. To help you better understand module imports, the following sections explore each of these steps in turn. First, though, bear in mind that all three of these steps are carried out only the first time a module is imported during a program's execution. Later imports of the same module in a program's run bypass all three of

these steps and simply fetch the already loaded module in memory. Python does this by storing loaded modules in a normal dictionary named `sys.modules`, and checking for a module's name there at the start of an import operation. In fact, `sys.modules['name']` means the same as `name` after an `import name`, and `sys.modules`' keys iterator or `keys` method lists all imported modules:

```
> >> import sys, os >> sys.modules['os'] is os # =>
True
> >> sorted(sys.modules) # sys.modules.keys()
... #
```

We'll explore other roles for `sys.modules` in upcoming chapters. On imports, though, if the requested module is not already present in `sys.modules`, a three-step process begins.

中文翻译

上一节谈到了导入模块，却没有完全解释导入时到底发生了什么。因为导入是 Python 程序结构的核心，本节将更正式地详述导入操作，让这个过程不再那么抽象。有些 C 程序员喜欢把 Python 的模块导入操作比作 C 的 `#include`，但他们真的不应该这么做——在 Python 中，导入不是把一个文件文本性地插入另一个文件。导入实际上是运行时（runtime）操作，在程序第一次导入某个文件时会执行三个不同的步骤：1. 查找（Find）模块的文件。2. 编译（Compile）为字节码（如果需要的话）。3. 运行（Run）模块的代

码，构建它定义的对象。为了帮你更好地理解模块导入，下面几个小节将依次探讨每一步。不过首先要记牢：这三个步骤只在程序执行期间第一次导入某个模块时才全部执行。之后对同一模块的再次导入会跳过这三步，只是取回内存中那个已加载的模块。Python 的做法是：把已加载的模块存进一个普通的字典 `sys.modules`，并在每次导入操作开始时先在这里检查模块名。事实上，在 `import name` 之后，`sys.modules['name']` 与 `name` 指向的就是同一个东西；而 `sys.modules` 的 `keys` 迭代器或 `keys` 方法会列出所有已导入的模块：

```
>>> import sys, os
>>> sys.modules['os'] is os      # =>
True
>>> sorted(sys.modules)          # sys.modules.keys()
...                               #
```

我们将在接下来的章节中探讨 `sys.modules` 的其他作用。但就导入而言，如果请求的模块尚不在 `sys.modules` 中，三步流程就开始了。

深度理解

- 核心概念：`import` 不是编译期文本粘贴，而是运行时的一次性三步操作（查找→编译→运行），重复导入只查缓存。理解 `import` 的关键就八个字：三步一次，命中即回。
- 底层实现：`import b` 在 CPython 中编译成 `IMPORTNAME b` 指令，调用底层的 `import()`：先查 `sys.modules` 字典，命中就直接返回模块对象；未命中才进入“查找 `spec` → 创建模块 → 执行代码 → 存入字典”的流程。`sys.mo`

dules['os'] is os 返回 True 就是最有力的证据——两者是同一个对象。

·设计原因：C 的 #include 在预处理阶段做文本粘贴，会导致同名重复定义、无法按需加载；Python 选择"运行时导入"，使 import 本身就是一条普通语句——可以写在函数里、条件分支里，实现延迟加载（lazy loading）。

·解决什么实际问题：大程序里多个文件相互引用，谁先 import 谁后 import 都不会乱——缓存保证模块代码只执行一次；也正因为"只执行一次"，写在模块顶层的副作用代码不会被反复触发。

·初学者误区：以为"再次 import 会重新执行模块代码"（不会，那是 importlib.reload 的事）；以为 import 语句必须出现在文件顶部（是通行规范，但技术上可以出现在任何位置）；以为 sys.modules 里存的是模块的"副本"（存的就是模块对象本身，is 判断为 True）。

代码分析

```
>>> import sys, os                                # sys  ossys os ...
>>> sys.modules['os'] is os                        # is
True
>>> sorted(sys.modules)                            #
['_abc', '_ast', '_codecs', ..., 'zipimport', 'os', 'sys']
```

解释器如何处理：

1. import sys, os: sys 直接命中内存缓存；os 不在缓存，于是走三步——在 sys.path 中找到 os.py、编译成字节码（或读既有 .pyc）、执行代码创建 os 模块对象，再把'os' 作为键存进 sys.modules。

2. `sys.modules['os']` is `os`: 从字典里取出的模块对象，与全局名字 `os` 绑定的对象是同一个，所以 `is` 返回 `True`——"字典里的名字就是模块本身"。

3. `sorted(sys.modules)`: 把字典的键排序打印——你会看到解释器已经偷偷加载的一大堆模块（`abc`、`zipimport` 之类），这正是"导入是运行时活动"的直接证据：连解释器自己都在动态加载。

设计思想：用"普通字典"做缓存而不是特殊机制，好处是任何代码都能查看甚至修改 `sys.modules`（例如敲 `sys.modules.pop('b')` 可强制让下一次 `import` 重新加载），一切透明、可调试。

22.7 Step 1: Find It (第一步：查找文件)

英文原文

First, Python must locate the module file referenced by an import statement. Notice that the import statement in the prior section's example names the file without a `.py` extension and without its directory path: it just says `import b`, instead of something like `import c:\dir\b.py` or similar on Unix. Path and extension details are omitted in imports on purpose; instead, Python uses a standard module search path along with known file types to locate the module file corresponding to an import statement. Because this is the main part of the import operation that programmers must kn

ow about, we'll return to this topic by itself in a moment.

中文翻译

首先, Python 必须定位 import 语句所引用的模块文件。请注意, 上一节例子中的 import 语句在命名文件时既不带 .py 扩展名, 也不带目录路径: 它只说 import b, 而不是像 import c:\dir\b.py 或 Unix 上类似的东西。路径和扩展名细节在导入中被故意省略; 取而代之, Python 使用标准的模块搜索路径 (module search path) 加上已知的文件类型, 来定位与 import 语句对应的模块文件。因为这是导入操作中程序员必须了解的主要部分, 我们稍后还会单独回头讲这个话题。

深度理解

- 核心概念: import 只给出"模块名", 文件在哪里、扩展名是什么, 统统交给搜索路径去解决。程序员唯一需要操心的就是: 搜索路径里有没有那里文件。
- 底层实现: 执行 import b 时, 解释器把名字 b 交给 PathFinder, 后者从左到右扫描 sys.path 里的每个目录, 寻找匹配的文件类型 (.py、.pyc、.so、同名目录等); 全部找不到就抛 ModuleNotFoundError。
- 设计原因: 省略路径和扩展名是为了可移植性——同一份代码在 Windows、Linux、macOS 上都能 import, 路径分隔符 (\ 与 /) 永远进不了 import 语句; 这也是"路径细节"与"导入逻辑"之间的职责分离。

·解决什么实际问题：你只需保证"要导入的文件在搜索路径里的某个目录中"，至于它在哪个平台、放在哪个层级目录，import 语句一概不用改。

·初学者误区：以为 import 时要写文件路径或扩展名（import b.py 直接语法错误）；以为模块名大小写不敏感（import B 与 import b 是两个完全不同的模块）；以为"找不到模块"是代码写错了而没往搜索路径上想——ModuleNotFoundError 八成是路径没配好。

22.8 Step 2: Compile It (Maybe) (第二步：编译（或许）)

英文原文

After finding a source code file that matches an import statement by traversing the module search path, Python next compiles it to a lower-level form known as bytecode, if necessary. We discussed bytecode in Chapter 2, but it's a bit richer than explained there. When you first import a module, Python compiles the module's .py source code file to bytecode, and saves the bytecode in a file with a .pyc extension if possible. On later program runs, Python will load the bytecode from its .pyc file and skip the compile step, as long as the bytecode file uses a compatible format and was made by the importing Python, and you have not edited and saved the source code file since the byteco

de file was made. Importantly, if you change a module's source code, its bytecode file will be re-created the next time you run a program that imports the module. This ensures that a module's bytecode is always in sync with its source, and your Python. All of this is automatic and can generally be taken on faith by most Python users, but a brief look at the complete story can help make the process less mysterious than it should be. In more detail, bytecode files can be created in two flavors, the first of which is used by default, and the second of which came online in Python 3.7: Timestamp-based bytecode files are created by simply importing modules and are the default, original, and most common option. This is the flavor to use if you want imports to be fast, and don't have atypical needs. Hash-based bytecode files are created by using the `compileall` or `pycompile` library modules. Once created, the Python command-line switch `--check-hash-based-pycs` may be used to configure their operation (see Python's manual for this flag's three options). In either model, Python automatically saves enough info to know when a bytecode file must be re-created. Specifically, bytecode files embed a "magic" number identifying the bytecode's format, along with either the source code file's last-modified timestamp and size, or a hash value derived from the source code file's content. Bytecode filenames also include the implementation name and version of the Python that created t

hem. When a module is imported, Python first checks to see if it was previously imported by the program, and uses the already loaded module if so. Otherwise, it looks for a usable bytecode file corresponding to the module's source code file, by comparing the info saved with the bytecode file against the current specs of the running Python and the source code file. A .pyc bytecode file is usable if it has the same base name as the source code file, and:- Uses a compatible format—by checking the "magic" number- Is up to date with the source code file—by comparing either saved timestamp and size, or hash value- Was created by the running Python—by inspecting implementation and version tags in the filename If there is a bytecode file that passes all these checks, it is loaded, and the compilation step is skipped. If not, the source code is compiled to bytecode, and either saved or resaved in a bytecode file with all the noted info. Programs still run if bytecode files cannot be saved (compiled code is then simply created in memory and discarded on exit), and the -B Python command-line switch can turn off bytecode saves, though it's rarely needed. When they are saved, bytecode files are written to and loaded from a subdirectory named pycache that's located alongside their corresponding source code files. The pycache subdirectory avoids both clutter in your source code folders, and contention and recompiles when multiple Pythons are installed. For example, here's w

hat happens when the same module file in this chapter's examples folder is imported by three different Pythons—two CPythons, and the PyPy we used in the preceding chapter (on Windows, use `dir` and `py` instead of `ls` and Python commands here):

```
$ ls
codefile.py
$ python3.12
>>> import codefile
$ python3.8
>>> import codefile
$ pypy3
>>> import codefile
$ ls
__pycache__ codefile.py

$ ls __pycache__
codefile.cpython-312.pyc codefile.cpython-38.pyc codefile.pypy310...
```

Technically, the `pycache` subdirectory was introduced in Python 3.2 and isn't available earlier, but this book is focused on Python 3.X only, and you're very unlikely to come across a 3.1 or 3.0 in the wild today. Also, keep in mind that bytecode compilation happens when a file is being imported. Because of this, you will not usually see a `.pyc` bytecode file for the top-level file of your program, unless it is also imported elsewhere—only imported files leave behind `.pyc` files on your machine. The bytecode of top-level files is used internally and discarded; bytecode of imported files is saved in files to speed up future imports. Top-level files are often designed to be executed directly and not i

mported at all. Later, though, you'll see that it is possible to design a file that serves both as the top-level code of a program and as a module of tools to be imported. Such a file may be either executed and imported and does generate a .pyc in the latter role. To learn how this works, watch for the discussion of the special name attribute and main in Chapter 25. NOTE: Running from bytecode only: As a now-special case, programs will also run if Python finds only bytecode files but no source code files, though this involves more than just deleting your .py files. As of Python 3.2, it generally requires that m.pyc files be located where m.py files normally would be—via either moving and renaming from pycache or generating with the legacy option of Python's compileall (-b in its command-line mode). For instance, the following makes .pyc files from all .py files in the current directory without requiring moves and renames:

```
$ python3 -m compileall -b -l .
```

Running from just bytecode requires a compatible Python and doesn't fully conceal your code, but is used by some tools to ship programs in standalone form when the hosting Python version can be included or ensured. See Python docs for more info, and the related solution in "Part I, Getting Started" in Appendix B.

中文翻译

在通过遍历模块搜索路径找到与 `import` 语句匹配的源代码文件之后，Python 接下来会在必要时把它编译成一种更低层的形式，称为字节码（bytecode）。我们在第 2 章讨论过字节码，但真实情况比那里讲的要丰富一些。当你第一次导入一个模块时，Python 会把模块的 `.py` 源代码文件编译成字节码，并在可能的情况下把字节码保存到扩展名为 `.pyc` 的文件中。在以后的程序运行中，只要字节码文件使用兼容的格式、由当前导入它的 Python 生成，而且自字节码文件生成以来你没有编辑并保存过源代码文件，Python 就会从 `.pyc` 文件加载字节码并跳过编译步骤。重要的是：如果你修改了模块的源代码，那么下一次运行导入该模块的程序时，它的字节码文件会被重新创建。这保证了模块的字节码始终与其源代码和你的 Python 保持同步。这一切都是自动的，大多数 Python 用户通常可以“信以为真”，但简单了解一下完整的故事，能让这个过程不再那么神秘。更详细地说，字节码文件有两种创建方式，第一种是默认的，第二种从 Python 3.7 起可用：基于时间戳（Timestamp-based）的字节码文件——通过简单导入模块即可创建，是默认的、原始的、最常见的选项。如果你希望导入速度快、又没有特殊需求，就用这种。基于哈希（Hash-based）的字节码文件——通过使用 `compileall` 或 `pycompile` 库模块创建。创建之后，可以用 Python 命令行开关 `--check-hash-based-pycs` 配置它们的运行方式（这个标志的三个选项请参见 Python 手册）。无论哪种模式，Python 都会自动保存足够的信息来判断字节码文件何时必须重新创建。具体来说，字节码文件内嵌一个标识字节码格式的“魔法数字（ma

gic number) ", 再附上源代码文件的最后修改时间戳和大小, 或者由源代码文件内容推导出的哈希值。字节码文件名还包含生成它的 Python 的实现名称和版本。导入模块时, Python 首先检查它是否已被程序导入过, 如果是就直接使用已加载的模块。否则, 它会查找与该模块源代码文件对应的可用字节码文件——把字节码文件中保存的信息与当前运行中的 Python 以及源代码文件的现状进行比较。一个 .pyc 字节码文件在满足以下条件时是可用的 (并且与源代码文件同名):

- 使用兼容的格式——通过检查"魔法数字"-
- 与源代码文件保持最新——通过比较保存的时间戳和大小, 或哈希值
- 由当前运行的 Python 创建——通过检查文件名中的实现与版本标签

如果存在一个通过全部检查的字节码文件, 就直接加载它, 跳过编译步骤。如果没有, 就把源代码编译成字节码, 并 (重新) 保存为带有所述全部信息的字节码文件。即使字节码文件无法保存, 程序也照常运行 (此时编译出的代码只是在内存中创建, 退出时丢弃);

- B 命令行开关可以关闭字节码保存, 但很少需要。

保存时, 字节码文件写入并加载自一个名为 pycache 的子目录, 它与对应的源代码文件位于同一位置。pycache 子目录既避免了源代码文件夹里的杂乱, 也避免了安装多个 Python 时的争用和重复编译。例如, 本章示例文件夹中的同一个模块文件被三种不同的 Python——两个 CPython 和上一章用过的 PyPy——导入时会发生下面这样的事 (在 Windows 上, 把 ls 换成 dir、把 python 命令换成 py 命令):

```
$ ls
codefile.py
$ python3.12
>>> import codefile
$ python3.8
>>> import codefile
$ pypy3
>>>> import codefile
$ ls
__pycache__ codefile.py

$ ls __pycache__
codefile.cpython-312.pyc codefile.cpython-38.pyc codefile.pypy310...
```

严格来说，`pycache` 子目录是 Python 3.2 引入的，更早的版本没有；但本书只关注 Python 3.X，而且你今天现实中遇到 3.1 或 3.0 的可能性极低。另外要记住：字节码编译发生在文件被导入时。正因如此，你通常看不到程序顶层文件的 `.pyc` 字节码文件，除非它也在别处被导入——只有被导入的文件才会你的机器上留下 `.pyc` 文件。顶层文件的字节码在内部使用后就被丢弃；被导入文件的字节码则保存成文件，以加速未来的导入。顶层文件通常被设计为直接执行、根本不被导入。不过后面你会看到，也可以把一个文件设计成“既是程序顶层代码，又是可导入的工具模块”。这样的文件既可以执行也可以导入，在后一种角色下它确实会产生 `.pyc` 文件。想了解这是怎么做到的，请留意第 25 章关于特殊属性 `name` 和 `main` 的讨论。注意：仅从字节码运行（Running from bytecode only）——作为一个相当特殊的案例：如果 Python 只找到字节码文件而找不到源代码文件，程序也能运行，不过这不仅仅是从你的电脑上删掉 `.py` 文件那么简单。从 Python 3.2 起，这通常要求 `m.pyc` 文件位于 `m.py` 文件通常所在的位置——要么从 `pycache`

移动并改名，要么用 Python compileall 的 legacy（传统）选项生成（命令行模式下的 -b）。例如，下面的命令把当前目录下所有 .py 文件都生成 .pyc 文件，而无需移动和改名：

```
$ python3 -m compileall -b -l .
```

仅凭字节码运行需要一个兼容的 Python，并且不能完全隐藏你的代码，但某些工具确实用它来以独立（standalone）形式交付程序——当宿主 Python 的版本可以被包含或确保时。更多信息见 Python 文档，以及附录 B 中"Part I, Getting Started"一节的对应解决方案。

深度理解

·核心概念：.pyc 是"导入的副产品"——它的唯一目的就是加速下一次进程的启动。三步流程中，第二次运行同一模块时被省掉的正是步骤 2；但要区分两种"第二次"：新进程里再导入（省编译），同一进程里再 import（连三步都省了，直接查 sys.modules）。

·底层实现："魔法数字"是每一版 Python 字节码格式的指纹（3.12 的魔法数字与 3.8 不同），对不上就拒绝加载、重新编译。文件名 codefile.cpython-312.pyc 的拆解：codefile（模块名）+ .cpython（实现名，CPython/PyPy 等）+ -312（版本）+ .pyc。时间戳模式比较"修改时间 + 大小"；哈希模式比较内容哈希，由 compileall/pycompile 生成，--check-hash-based-pycs 的三个选项分别是 default / always / never。

·设计原因：为什么把编译产物放进 pycache 子目录而不

是源文件旁边？一是避免弄脏源码目录；二是多版本 Python 共存时互不覆盖（cpython-312、pypy310 各存各的），避免"一个 .pyc 被另一个版本覆盖导致反复重编译"的争用。

·解决什么实际问题：程序第二次启动明显变快（跳过编译）；交付时想减少源代码暴露或加速加载，可以只分发 .pyc（compileall -b，需保证宿主 Python 版本匹配）；-B 开关在只读文件系统上避免写 .pyc 失败报错。

·初学者误区：以为 .pyc 是"编译后的机器码"或"反向工程难产的加密产物"（它只是字节码，几乎等于源码的"易读翻译版"）；以为删掉 .pyc 会丢功能（下次导入自动重建）；以为顶层脚本也会生成 .pyc（只有被导入的文件才会留文件）；以为"改了源码程序还走旧逻辑"是缓存坏了（时间戳/哈希检查会自动使缓存失效并重编）。

代码分析

```
$ ls #
codefile.py
$ python3.12
>>> import codefile # CPython 3.12 ...
$ python3.8
>>> import codefile # CPython 3.8 ...
$ pypy3
>>>> import codefile # PyPy 3.10 ...
$ ls
__pycache__ codefile.py #
$ ls __pycache__
codefile.cpython-312.pyc codefile.cpython-38.pyc codefile.pypy310...
```

解释器如何处理：

1. 每个 Python 进程第一次 import codefile 时，解释器在 sys.path 中定位到 codefile.py，发现还没有可用的 .pyc，于是编译源码、生成字节码。
2. 写文件时，解释器用"实现名 + 版本号"构造缓存文件名（cpython-312、cpython-38、pypy310），放进源码旁的 pycache 目录——三个解释器互不覆盖。
3. 下次（新的进程）再导入时，解释器比对魔法数字、时间戳/哈希和文件名标签，全部通过就直接加载 .pyc、跳过编译，启动更快。

设计思想：把"编译一次、到处缓存"做成完全自动的机制，用户零感知；多版本并存时的命名隔离，让"一台机器装多个 Python"成为常态而不是灾难。

22.9 Step 3: Run It (第三步：运行)

英文原文

After compiling or loading the module's bytecode, the final step of an import operation executes the byte code. This effectively runs all the statements in the module's file in turn, from top to bottom, and any assignments made to names during this step generate attributes of the resulting module object. This is how the tools defined by the module's code are created. For instance, def statements in a file are run at import

time to create functions and assign attributes within the module to those functions. The functions can then be called later in the program by the file's importers. Because this last import step actually runs the file's code, if any top-level code in a module file does real work, you'll see its results at import time. For example, top-level print statements in a module show output when the file is imported. Function def statements (and class statements up later in this book) simply define objects for later use. As you can see, import operations involve quite a bit of work—they search for files, possibly run a compiler, and run Python code. Because of this, any given module is imported only once per process by default. Future imports skip all three import steps and reuse the already loaded module in memory, per the `sys.modules` check noted earlier. If you need to import a file again after it has already been loaded (for example, to support dynamic customizations), you can force the issue with an `importlib.reload` call—a tool we'll study in the next chapter. Bear in mind that this process is completely automatic—it's a side effect of running programs—and most programmers probably won't care about or even notice the mechanics, apart from faster startups due to skipped compile steps. One part of this process is likely to show up on your radar, though, per the next section's elaboration.

中文翻译

在编译或加载模块的字节码之后，导入操作的最后一步是执行字节码。这实际上就是把模块文件里的所有语句从上到下逐条运行一遍，而这一步中对名字的任何赋值都会生成最终模块对象的属性。模块代码所定义的工具就是这样被创建的。例如，文件中的 `def` 语句在导入时被执行，创建函数并把模块内的属性指向这些函数。之后，文件的外部导入方就可以在程序中调用这些函数了。因为导入的最后一步真的会运行文件代码，所以如果模块文件中的任何顶层代码做了实际工作，你会在导入时看到它的结果。例如，模块中的顶层 `print` 语句会在文件被导入时打印输出。函数 `def` 语句（以及本书后面出现的 `class` 语句）则只是定义对象供以后使用。如你所见，导入操作涉及不少工作——搜索文件、可能运行编译器、运行 Python 代码。正因如此，默认情况下任何模块每个进程只导入一次。后续导入会跳过全部三步，按照前面提到的 `sys.modules` 检查，复用内存中已加载的模块。如果你需要在文件加载后再次导入它（例如为了支持动态定制），可以用 `importlib.reload` 调用强制实现——这是我们下一章要研究的工具。请记住，这个过程完全是自动的——它是运行程序的副产物——大多数程序员可能根本不会在意甚至注意不到这些机制，能感觉到的只是“跳过编译后启动更快”而已。不过，这个过程上的某一部分很可能会进入你的视野，正如下一节的展开所述。

深度理解

·核心概念：`import` 的第三步就是“执行”，模块的一切内容（函数、类、变量）都是在导入瞬间真正“运行”出来的

。def 是语句，import 时执行它才产生函数对象。

·底层实现：模块字节码在模块对象自带的帧（frame）里执行，作用域就是模块的全局命名空间；每一条赋值指令（STORENAME）直接往模块的 dict 里写属性。这就是“全局作用域变身属性命名空间”的机械过程。

·设计原因：为什么导入要“真的运行代码”而不是“扫描函数签名”？因为 Python 是动态语言，模块内容可以是运行时计算出来的（例如根据条件生成不同的函数），只有执行才知道模块里有什么。

·解决什么实际问题：正因“导入即执行”，模块文件必须保持“顶层只定义、不做事”的卫生习惯；否则一导入就打印、就连网、就写文件，导入方会被副作用淹没。要重跑模块代码（热更新配置、调试），用 importlib.reload。

·初学者误区：以为 def 只是“登记了个名字”（其实 import 时它会真的创建函数对象）；以为模块被导入两次会执行两次顶层代码（不会，缓存保证一次）；以为“模块里 print 不出来”是坏了（把 print 放进函数体就不会打印——那是设计使然）；以为 reload 之后旧对象全部自动更新（from b import x 拿到的旧引用不会自动变，见第 23 章）。

代码分析

```

# m.py
print('m ')          # print
x = 100               # m.x
def add(a, b):        # def
    return a + b

# main.py
import m              # m
print(m.x)            # 100
print(m.add(1, 2))    # 3

```

解释器如何处理：

1. `import m` 执行到第三步时，解释器在 `m` 模块对象的命名空间中逐条运行 `m.py` 的字节码。
2. `print('m 被加载了')` 立即执行，终端输出；`x = 100` 执行 `STORENAME`，把属性 `x` 写进模块字典；`def add` 执行 `MAKEFUNCTION + STORENAME`，创建函数对象并绑定为属性 `add`。
3. 主脚本随后通过 `m.x`、`m.add(1, 2)` 属性访问拿到这些工具——它们此刻才真正存在。

设计思想："模块内容 = 执行结果"这一设计把模块与函数统一起来：函数体是"调用时执行"，模块体是"导入时执行"，机制完全一致——所以模块又叫"顶层的代码"。

22.10 The Module Search Path (模块搜索路径)

英文原文

As mentioned earlier, the part of the import procedure that most programmers will need to care about is usually the first "find it" part—locating the file to be imported. Because you may need to tell Python where to look to find files to import, you need to know how to tap into its module search path (the set of directories searched) in order to extend it. This section covers the components that make up the search path and describes how to mod it for folders of your own. Special case: built-in modules like `sys`, coded in C and statically linked into Python, as well as frozen modules like `os`, optimized for faster startup as of Python 3.11, are always checked first before scanning the module search path and hence take precedence. Given that there are only a few standard-library modules in these categories, we can safely ignore them here and focus on the search used for the vast majority of modules you'll import—including your own. If you're curious, `sys.builtinmodulenames` lists the items in the first of these extrasearch categories:

```
>>> sum(1 for m in sys.builtin_module_names if m[0] != '_')    #...
11
```

中文翻译

如前所述，大多数程序员需要关心的导入环节通常是第一步“查找文件”——定位要导入的文件。因为你可能需要告诉 P

Python 去哪里找要导入的文件，所以必须知道如何接入它的模块搜索路径（module search path）（被搜索的目录集合）以便扩展它。本节讲解组成搜索路径的各部分，并描述如何为你的文件夹改造它。一个特例：内建模块（built-in module）如 `sys`——用 C 编写并被静态链接进 Python——以及冻结模块（frozen module）如 `os`——从 Python 3.11 起为加速启动做了优化——总是在扫描模块搜索路径之前被优先检查，因而优先级更高。既然这类标准库模块只有那么几个，我们可以放心忽略它们，重点关注你要导入的绝大多数模块（包括你自己的模块）所使用的搜索机制。好奇的话，`sys.builtinmodulenames` 会列出第一类额外搜索项的成员：

```
>>> sum(1 for m in sys.builtin_module_names if m[0] != '_')    #...
11
```

深度理解

- 核心概念：模块搜索路径 = 一组按顺序查找的目录。导入大部分模块都要靠它，只有极少数“内建/冻结”模块能插队优先。
- 底层实现：内置模块是编译 CPython 时静态链接进解释器的（如 `sys`、`time`），永远在内存里；冻结模块（如 3.11 起的 `os`、3.12 起的 `sysconfig` 等）把源码编译进解释器镜像，省去文件 IO，加快启动。`sys.builtinmodulenames` 返回的就是第一类系统内建模块的名字元组。
- 设计原因：把最常用、最基本的模块“焊死”进解释器，是为了让核心功能不依赖文件系统——即使 `sys.path` 被破坏，`sys` 也永远可用。

·解决什么实际问题：解释器自举 (bootstrap) 依赖 sys ; 解释器在没找到任何库目录时也照样能 import 内建模块，完成最基本的启动。

·初学者误区：以为"所有模块都要在搜索路径里"（内建/冻结就不需要）；以为 Python 3.11 之后 os 是 C 写的（它仍是纯 Python，只是被冻结进了解释器）；以为 sys.builtinmodulenames 很长（去掉下划线开头的其实只有 11 个）。

22.11 Search-Path Components (搜索路径的组成部分)

英文原文

In many cases, you can rely on the automatic nature of the module search path and won't need to configure this path at all. If you want to be able to import user-defined files across directory boundaries, though, you will need to customize this path. Roughly, Python's module search path is composed of the concatenation of the following components, some of which are preset for you and some of which you can tailor to tell Python where to look: 1. The home directory of the program 2. Directories listed in PYTHONPATH (if set) 3. Standard-library directories and files 4. The site-packages directory of third-party extensions 5. The directories listed in any .pth files (if present) Ultimate

ly, the concatenation of these five components initializes the built-in `sys.path`—a changeable list of directory-name strings that are searched from first to last for imported files (we'll revisit this later in this section). The first, third, and fourth components of the search path are defined automatically. The second and fifth components, though, can be used to extend the path to include your own source code directories. Here's a rundown on all five: Home directory (automatic) — Python first looks for the imported file in the code's "home" directory. The meaning of this entry depends on how you are running the code. When you're running a program, this entry is the directory containing your program's top-level script file. When you're working interactively in a REPL, this entry is instead the directory in which you are currently working (which is why we've used this directory for imported files so far). Code run with Python's `-c` and `-m` switches used in earlier chapters also uses the current working directory for the home component. Because this directory is always searched first, if a program is located entirely in a single directory, all of its imports will work automatically with no path configuration required. On the other hand, because this directory is searched first, its files will also override modules of the same name in directories elsewhere on the path; be careful not to accidentally hide standard-library modules this way if you need them in your program, or use package tools y

ou'll meet later that can partially sidestep this issue with nested folders. Also remember that this home directory pertains only to the search path used to resolve imports, and does not affect the current working directory used for relative filenames in your script. Pathless files created by a script will still show up where you are when you launch it (specifically, in the folder returned by `os.getcwd()`), not in the script's home folder at the front of the module search path. Module imports and file access are disjoint ideas. Any `PYTHONPATH` directories (configurable) — Next, Python searches all directories listed in your `PYTHONPATH` environment variable setting, from left to right (assuming you have set this at all: it's not generally preset for you). In brief, `PYTHONPATH` is simply a clearly named list of user-defined and platform-specific directory names that host Python source code or bytecode files. You can add all the directories from which you wish to be able to import, and Python will extend the module search path to include all the directories listed in your `PYTHONPATH`. See Appendix A for tips on setting this variable. Because Python searches the home directory first, this setting is only important when importing files across directory boundaries—that is, if you need to import a file that is stored in a different directory from the file that imports it. You may need to set your `PYTHONPATH` variable once you start writing substantial programs and tools, but when you're first starting

out, as long as you save all your module files in the directory in which you're working (i.e., the home directory) your imports will work without making this setting. Standard-library directories (automatic) — Next, Python automatically searches the directories where the standard-library modules are installed on your machine. These include modules coded in Python, and others coded in C. Because these directories are always searched, they normally do not need to be added to your PYTHONPATH. The standard-library site-packages directory of third-party extensions (automatic) — Next, Python automatically adds the site-packages subdirectory of its standard library to the module search path. By convention, this is the place where most third-party extensions are installed, often automatically by Python's pip install tool. Because the site-packages install directory of such extensions is always part of the module search path, clients can import the modules of these extensions with no path settings at all. Any .pth path-file directories (configurable) — Finally, a lesser-used feature of Python allows users to add directories to the module search path by simply listing them, one per line, in a text file whose name ends with a .pth suffix (for "path"). These path-configuration files are a somewhat advanced installation-related feature; we won't cover them fully here, but they provide an alternative to PYTHONPATH settings. In short, text files of directory names dropped in an appropriate

directory can serve roughly the same role as the PYTHONPATH environment variable setting. For instance, a file with .pth extension and any name may be placed in the site-packages subdirectory of the installed Python's standard library to extend the module search path. To locate this subdirectory inspect sys.path for a path ending in site-packages, per its coverage ahead. When such a file is present, Python will add the directories listed on each line of the file, from first to last, after the site-packages directory in the module search path. In fact, Python will collect the directory names in all the .pth path files it finds and filter out any duplicates and nonexistent entries. Because they are files rather than shell settings, path files can apply to all users of a given Python, instead of just one user, and may be simpler to set up than environment variables in some contexts. For more details on this file feature, consult Python's library manual, and especially the module named site—the standard-library module that handles the search-path process, configuring the locations of Python's libraries and path files. When getting started, though, you may be better off setting PYTHONPATH, and only if you must import across directories. Path files are used more often by third-party library installers. All that being said, Python's import mechanism is wildly extensible, and even more convoluted than described here. For example, PYTHONPATH entries may also name ZIP files treated like read-o

nly folders; PYTHONHOME can set standard-library locations; a file named with suffix .pth can fully override sys.path norms; "virtual environments" made with the module venv avoid package version clashes by localizing search paths to install folders; the relative-import "." syntax of Chapter 24 constrains module search in packages; and all this has morphed regularly, and may again. While this book covers common usage, its description is intentionally narrow for space, durability, and audience. You should consult Python's manuals for more details on the imports story if and when unique requirements arise.

中文翻译

很多情况下，你可以依赖模块搜索路径的自动机制，根本不需要配置它。但如果想在跨目录的边界导入自己写的文件，你就需要定制这个路径了。粗略地说，Python 的模块搜索路径由以下部分的拼接而成，其中一些是系统为你预设的，另一些可以由你定制，告诉 Python 去哪里找：1. 程序的主目录 (home directory) 2. PYTHONPATH 中列出的目录 (如果设置了) 3. 标准库目录和文件 4. 第三方扩展的 site-packages 目录 5. 任何 .pth 文件中列出的目录 (如果存在) 归根结底，这五个部分的拼接初始化了内建的 sys.path——一个可修改的目录名字符串列表，导入时按从前到后的顺序搜索 (本节后面我们会再看它)。搜索路径的第一、三、四部分自动定义。第二和第五部分则可以用来扩展路径，把你的源码目录包含进来。下面逐一说明这五部分：主目录 (自动) —— Python 首先在代码的 "主目录" 中查找要导

入的文件。这一项的含义取决于你怎么运行代码：运行程序时，它是包含程序顶层脚本文件的目录；在 REPL 中交互式工作时，它是你当前工作的目录（这也是我们此前一直用这个目录放置被导入文件的原因）。在更早章节用到的 `-c` 和 `-m` 开关运行的代码，其主目录那一项同样取当前工作目录。因为这个目录总是最先被搜索，如果一个程序整个落在单个目录里，它的所有导入都会自动工作，无需任何路径配置。另一方面，正因为这个目录最先被搜索，它的文件也会覆盖（override）路径上其他目录里的同名模块；如果你需要标准库模块，要当心别用这种方式意外地把它们藏起来——除非使用后面会遇到的、能部分借助嵌套文件夹规避此问题的包工具。还要记住：这个主目录只作用于解析导入所用的搜索路径，并不影响脚本中相对文件名所用的当前工作目录。脚本创建的无路径文件仍然出现在你启动它的地方（具体来说，是 `os.getcwd` 返回的文件夹），而不是出现在模块搜索路径最前端的脚本主目录里。模块导入与文件访问是两码事。任何 `PYTHONPATH` 目录（可配置）——接下来，Python 会从左到右搜索你的 `PYTHONPATH` 环境变量设置中列出的所有目录（前提是你确实设置了它：通常它并没有帮你预设）。简单说，`PYTHONPATH` 就是一个用户自定义的、与平台相关的目录名列表，这些目录存放 Python 源代码或字节码文件。你可以把想要从中导入的所有目录都加进去，Python 会把 `PYTHONPATH` 中列出的所有目录扩展进模块搜索路径。关于设置这个小技巧见附录 A。因为 Python 先搜索主目录，这个设置只有在跨目录导入时才重要——也就是说，当你需要导入一个“与导入它的文件不在同一目录”的文件时。一旦开始写大型程序和工具，你可能需要设置 `PYTHONPATH`；但刚开始学习时，只

要把所有模块文件都保存在你工作的目录（即主目录）里，不设置它导入也能正常工作。标准库目录（自动）——接着，Python 自动搜索你机器上安装标准库模块的目录。其中既有用 Python 写的模块，也有用 C 写的。因为这些目录永远会被搜索，所以通常不需要把它们加进 PYTHONPATH。第三方扩展的标准库 site-packages 目录（自动）——然后，Python 自动把标准库下的 site-packages 子目录加入模块搜索路径。按惯例这是大多数第三方扩展的安装位置，通常由 Python 的 pip 安装工具自动完成。因为这类扩展的 site-packages 安装目录永远是模块搜索路径的一部分，客户端根本不需要任何路径设置就能导入这些扩展的模块。任何 .pth 路径文件目录（可配置）——最后，Python 有一个较少用到的特性：允许用户在文件名以 .pth 后缀（path 的缩写）结尾的文本文件里每行列出一个目录，从而把目录加入模块搜索路径。这些路径配置文件是较高级的、与安装相关的特性；这里不全面讲解它们，但它们提供了 PYTHONPATH 设置之外的另一种选择。简而言之，把写有目录名的文本文件放进合适的目录，就能基本替代 PYTHONPATH 环境变量的作用。比如，任何名字的 .pth 后缀文件都可以放在已安装 Python 标准库的 site-packages 子目录里来扩展模块搜索路径。要找到这个子目录，可以在 sys.path 里找一个以 site-packages 结尾的路径，正如后面的讲述。存在这样的文件时，Python 会把文件中每一行列出的目录按从上到下的顺序，添加到模块搜索路径中 site-packages 目录之后。事实上，Python 会把找到的全部 .pth 路径文件里的目录名收集起来，并过滤掉重复和不存在的条目。因为它们是文件而不是 shell 设置，路径文件能作用于某个已安装 Python 的所有用户，而不是只影响一个用

户；在某些场景下比环境变量更好配。关于这个特性的更多细节，请查阅 Python 库手册，尤其是名为 `site` 的模块——这个标准库模块负责处理搜索路径，配置 Python 的安装和路径文件的位置。不过，刚起步时，你可能先用设置 `PYTHONPATH` 的方式比较好，而且只有当必须跨目录导入时才需要。路径文件更多地被安装在 `site-packages` 里的第三方库使用。话说回来，Python 的导入机制其实是高度可扩展的，甚至比这里描述的还要曲折。例如：`PYTHONPATH` 条目也可以指向被当作只读文件夹的 ZIP 文件；`PYTHONHOME` 可以设置标准库的位置；后缀为 `.pth` 的文件可以完全覆盖 `sys.path` 的规范；用 `venv` 模块创建的“虚拟环境（virtual environment）”通过把搜索路径局限到安装目录来避免包版本冲突；第 24 章讲的相对导入 `“.”` 语法会限制包内搜索；这一切都在不断演变，将来还可能变化。虽然本书涵盖常见用法，但为了避免空间、耐久性和目标读者的考虑，其描述刻意“窄”。当独特需求出现时，还是应该查阅 Python 文档中更多关于导入的细节。

深度理解

- 核心概念：搜索路径 = 主目录 + `PYTHONPATH` + 标准库 + `site-packages` + `.pth` 五部分拼接。前三个自动，后两个可定制；五部分最终体现为 `sys.path` 列表。
- 底层实现：解释器启动时通过 `site` 模块构建 `sys.path`：把脚本目录（或当前目录）放前面，再读 `PYTHONPATH`（用 `os.pathsep` 分隔目录）、追加标准库目录和 `site-packages`，最后处理 `.pth` 文件。ZIP 导入由 `zipimport` 支持，可把 `.zip` 当作目录看待——这也是 `sys.path` 里常

见 python312.zip 的原因。

·设计原因：为什么设计了"主目录第一优先"？为了让"单目录小项目零配置"成为默认体验。为什么保留这么多可定制入口（PYTHONPATH、.pth、pth 覆盖、PYTHONHOME、venv、ZIP）？为适应从"个人实验"到"企业级部署"的全谱系需求。

·解决什么实际问题：跨目录引用模块（PYTHONPATH）；第三方库安装后不用配路径（site-packages）；不污染 shell 配置就把路径加给所有用户（.pth）；避免多项目依赖版本冲突（venv）。

·初学者误区：以为所有项目都要配 PYTHONPATH（单目录根本不需要）；以为"模块导入路径"和"文件读写路径"是一回事（import 用 sys.path，开文件用当前工作目录）；以为 .pth 文件只能放 site-packages（那是约定位置，不是硬性语法）；以为主目录永远是脚本所在目录（REPL 里是当前工作目录）。

代码分析

五部分	是否自动	用途	生活类比
主目录 (Home directory)	自动	顶层脚本所在目录 / REPL 当前	家就在那里，直接回家
PYTHONPATH	可配置	用户指定的额外源码/字节码目	手机里的"常用地址清单"
标准库目录	自动	安装 Python 时自带的库	政府统一规划的公共设施
site-packages	自动	pip 安装的第三方扩展	楼盘的"临街商铺区"
.pth 文件	可配置	以文件形式追加的路径	快递柜里的"新地址卡片"

解释器如何处理：启动时 `site.py` 依次执行——把顶层脚本目录插入 `sys.path[0]` → 解析 `PYTHONPATH` 环境变量并逐个追加 → 追加标准库目录 → 追加 `site-packages` → 扫描 `.pth` 文件。整个过程是"拼接"，顺序即优先级：越靠前的目录越先被搜索，同名模块优先取最左边。

22.12 Configuring the Search Path (配置搜索路径)

英文原文

The net effect of all of the foregoing is that both the `PYTHONPATH` and path file components of the search path allow you to tailor the places where imports look for files. The way you set environment variables and where you store path files varies per platform. For instance, on macOS and Windows, you might set `PYTHONPATH` to a list of directories separated by colons and semicolons, respectively, like this:

```
/Users/me/pycode/utilities:/Volumes/ssd/pycode/package1 # Unix 路径列表 (冒号分隔)
C:\Users\me\pycode\utilities;d:\pycode\package1 # Windows 路径列表 (分号分隔)
```

On Unix systems (e.g., macOS, Linux, and Android), an export shell command sets environment variables for the shell and everything it runs, and can be coded in startup files like `~/.bashprofile` for permanence. Th

e following on macOS sets the path to enable imports of modules from the code folders of Chapters 21 and 20 ("..." is snipped text):

```
python $ pwd /Users/me/.../LP6E/Chapter22
$ export PYTHONPATH=/Users/me/.../LP6E/Chapter21:/Users/me/.../LP6E/Chapter20 $ python3 >>> import pybench, permute >>> pybench <module'pybench' from'/Users/me/.../LP6E/Chapter21/pybench.py'>
>>> permute <module'permute' from'/Users/me/.../LP6E/Chapter20/permute.py'>
```

Path syntax like ".." for parent folders also works on PYTHONPATH and is interpreted relative to the current working directory—which is not necessarily the home directory of launched programs, if their top-level scripts live elsewhere. Windows consoles use similar commands (e.g., replace export with set, or use setx for permanence), but Windows users generally set environment variables in Settings; search for the environment-variables GUI there. Instead of—or in addition to—PYTHONPATH, you might create a text file in your Python install's site-packages folder named with a .pth extension (e.g., mypath.pth), which looks like this on macOS:

```
/Users/me/pycode/utilities # Unix 版.pth 路径文件 /Volumes/ssd/pycode/package1 # 每行一个路径
```

These settings are analogous on all platforms, but th

e details vary too widely for fully inclusive coverage here. See Appendix A for pointers on extending your module search path with PYTHONPATH on various platforms. To see how your Python configures the module search path on your platform, and to find the location of its site-packages folder which can host path files, you can always inspect `sys.path`—the topic of the next section.

中文翻译

上述一切的净效果是：搜索路径中的 PYTHONPATH 和路径文件这两块，让你能够定制导入到哪里找文件。环境变量的设置方式、路径文件的存放位置，都因平台而异。例如，在 macOS 和 Windows 上，你可能分别用冒号和分号把目录列表设置进 PYTHONPATH，像这样： /Users/me/pycode/utilities:/Volumes/ssd/pycode/package1 # Unix 路径列表（冒号分隔） C:\Users\me\pycode\utilities;d:\pycode\package1 # Windows 路径列表（分号分隔）在 Unix 系统（如 macOS、Linux 和 Android）上，export shell 命令为 shell 及其运行的任何东西设置环境变量，并且可以写进 ~/.bashprofile 之类的启动文件以获得永久效果。下面这段 macOS 命令把路径设置为允许从第 21 章和第 20 章的代码文件夹导入模块（"..."是省略的文本）：python \$ pwd /Users/me/.../LP6E/Chapter22 \$ export PYTHONPATH=/Users/me/.../LP6E/Chapter21:/Users/me/.../LP6E/Chapter20 \$ python3 >>> import pybench, permute >>> pybench <module'pybench' from '/Users/me/.../LP6E/Chapter21/pybench.py'> >>> permu

te <module'permute' from'/Users/me/.../LP6E/Chapter20/permute.py'> 像 ".." 这种访问父文件夹的路径语法同样适用于 PYTHONPATH，并且是相对于当前工作目录解释的——如果顶层脚本在别处，这个目录不一定是所运行程序的主目录。Windows 控制台使用类似的命令（比如把 export 换成 set，或用 setx 获得永久性），但 Windows 用户通常是在“设置”里配置环境变量；在那里搜索环境变量图形界面即可。除了 PYTHONPATH——或者作为它的补充——你也可以在 Python 安装的 site-packages 文件夹里创建一个以 .pth 结尾的文本文件（例如 mypath.pth），它在 macOS 上长这样：/Users/me/pycode/utilities # Unix 版 .pth 路径文件/Volumes/ssd/pycode/package1 # 每行一个路径这些设置在各个平台上是类似的，但细节差异太大，这里无法全覆盖。关于在各种平台上用 PYTHONPATH 扩展模块搜索路径的指引，见附录 A。想看看你的 Python 在你的平台上是如何配置模块搜索路径的，并找到那个可以放路径文件的 site-packages 文件夹的位置，你随时可以检查 sys.path——这正是下一节的主题。

深度理解

·核心概念：配置搜索路径的两种主要手段已经齐了：环境变量 PYTHONPATH 与 .pth 文件；两者的差别是 y 一个按“用户”生效，一个按“安装的 Python”生效。

·底层实现：macOS/Unix 用 export、~/.bashprofile 持久化；Windows 用 set/setx 或图形界面。目录分隔符 Unix 用 :，Windows 用 ;——正是 os.pathsep 的值，别记反。

·设计原因：让模块导入与代码本身解耦——路径属于运行环境，不属于源码。这样同一份代码在不同机器/不同用户下可以指向不同版本的库。

·解决什么实际问题：命令行的 `import pybench, permute` 能直接成功；演示了"召回"旧章节的工具模块；CI/CD 环境里用环境变量注入依赖路径。

·初学者误区：以为 `PYTHONPATH` 一旦设置永远生效（每个新 shell 都要重新 `export`，除非写进 profile）；以为路径里不能有相对路径`".."`（可以，且相对当前工作目录）；以为环境变量改了立刻对已运行的解释器生效（必须在启动 Python 之前设置）。

22.13 The `sys.path` List (`sys.path` 列表)

英文原文

If you want to see how the module search path is truly configured on your machine, print the built-in `sys.path` list. This list of directory-name strings is the actual module search path within Python; on imports, Python searches each directory in this list from left to right and uses the first file match it finds. Python configures `sys.path` at program startup, merging the path components we met earlier. The result is a list of directories searched on each import of a new file. Python exposes this list for multiple reasons. For one thing, it provides a way to verify the search-path settings you'

ve made—if you don't see your settings somewhere in this list after restarting Python, you need to recheck your work. On macOS, for example, with these custom settings: - PYTHONPATH set to /Users/me/pycode1:/Users/me/pycode2 - A mypath.pth path file in the Python install's site-packages that lists /Users/me/pycode3 The search path looks like this when inspected in a REPL or printed from a script:

```
> >> import sys >> sys.path

['', '/Users/me/pycode1', '/Users/me/pycode2',
 '/Library/Frameworks/Python.framework/Versions/3.12/lib/python312...
 '/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.1...
 '/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.1...
 '/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.1...
 '/Users/me/pycode3']
```

The empty string at the front means the current directory. The two custom settings are merged in per the order given earlier—PYTHONPATH entries show up first, and then settings from the .pth path file. The rest are standard-library folders and files, plus the site-packages root directory for third-party extensions. The sys.path list also provides a way for scripts to tailor their search paths manually at runtime. By modifying the program-wide sys.path list, you modify the search path for all future imports made anywhere in a program's run. Such changes last only for the duration of the single run, however; PYTHONPATH and .pth files offer more permanent ways to modify the path—the first per user, and the second per installed Python. On

the other hand, some programs really do need to change `sys.path`. Scripts that run on web servers, for example, often run as the user "nobody" to limit machine access. Because such scripts cannot usually depend on "nobody" to have set `PYTHONPATH` in any particular way, they sometimes set `sys.path` manually to include required source directories, prior to running `import` statements. A `sys.path.append`, `sys.path.insert`, or other list operation will often suffice, though will endure for a single program run only:

```
> >> sys.path.append('/Users/me/pycode4') # >> impo...
```

You can mod `sys.path` arbitrarily in code to influence future imports, and this also works in a REPL. Bear in mind, though, that deleting folders may remove access to important tools, and your changes will be discarded when the program or REPL ends. To make path changes span scripts and sessions, use other techniques instead.

中文翻译

如果你想看看你的机器上模块搜索路径究竟是怎么配置的，把内建的 `sys.path` 列表打印出来即可。这个目录名字字符串列表就是 Python 内部真正的模块搜索路径；导入时，Python 从左到右搜索这个列表里的每个目录，并使用找到的第一个匹配文件。检查模块搜索路径（Inspecting the module search path）——Python 在程序启动时配置 `sys.path`，把我们前面遇到的路径组成部分合并起来。结果就是每次导入新文件都会搜索的目录列表。Python 公开这个列

表有多个原因。其中之一是提供一种验证 (verify) 你所做搜索路径设置的方法——如果你在重启 Python 后没有在这个列表里的某个位置看到自己的设置，就需要复查你的工作。例如在 macOS 上，使用这些自定义设置时：- PYTHONPATH 设为 /Users/me/pycode1:/Users/me/pycode2- Python 安装的 site-packages 里放着一个 mypath.pth 路径文件，其中列出了 /Users/me/pycode3在 REPL 里检查或从脚本里打印出来时，搜索路径长这个样子：

```
>>> import sys
>>> sys.path
['', '/Users/me/pycode1', '/Users/me/pycode2',
 '/Library/Frameworks/Python.framework/Versions/3.12/lib/python312...
 '/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.1...
 '/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.1...
 '/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.1...
 '/Users/me/pycode3']
```

最前面的空字符串表示当前目录。两个自定义设置按照前面给出的顺序并入——PYTHONPATH 的条目排在前，随后是 .pth 路径文件带来的设置。其余是标准库文件夹和文件，以及存放第三方扩展的 site-packages 根目录。sys.path 列表也为脚本提供了一种在运行时手动定制 (tailor) 搜索路径的方式。通过修改程序级的 sys.path 列表，你就修改了程序运行期间任意位置所有未来导入的搜索路径。不过，这样的改动只持续到单次运行结束；PYTHONPATH 和 .pth 文件是更持久的修改方式——前者按用户生效，后者按已安装的 Python 生效。另一方面，有些程序确实需要修改 sys.path。比如运行在 Web 服务器上的脚本——它们常常以用户 "nobody" 的身份运行，以限制对机器的访问。因为这类脚本通常不能指望 "nobody" 以某种特定方式设置过 PYTHONPATH (语法上不正确，但事实如此)，它们

有时会在运行 import 语句之前手动设置 sys.path，以包含所需的源码目录。sys.path.append、sys.path.insert 或其他列表操作通常就够用了，但只对单次程序运行有效：

```
>>> sys.path.append('/Users/me/pycode4')    #  
>>> import module                            #
```

你可以在代码里随意修改 sys.path 去影响未来的导入，这在 REPL 里同样有效。不过要记住：删除文件夹可能会让你失去对重要工具的访问权限，而且程序或 REPL 结束时，你的修改就会被丢弃。要让路径修改跨脚本、跨会话生效，就用其他技术。

深度理解

- 核心概念：sys.path 就是把五个组成部分拼接后的"成品列表"，既能查看（验证配置），也能运行时修改（按需定制）。

- 底层实现：sys.path 是解释器 C 层维护的可变列表对象，append/insert 直接修改它。首元素"在 CPython 里被解释为"当前工作目录"；启动脚本时解释器还会把脚本所在目录插入 sys.path[0]（python -c 或 REPL 则插入当前目录）。

- 设计原因：把配置暴露为普通可改列表而不是只读快照，是为了给"运行时动态挂载目录"留活口——很多框架和插件系统就是在启动代码里直接操纵 sys.path。

- 解决什么实际问题：Web 服务器以 "nobody" 运行、无法依赖环境变量时，脚本自行 append 路径；验证配置（打印 sys.path 确认设置生效）；临时挂一个目录做实验

，跑完即弃。

·初学者误区：以为 `sys.path[0]` 的空字符串"是错误"（它就是当前目录）；以为修改 `sys.path` 是永久的（只活到本次进程结束）；以为删掉 `sys.path` 里的目录只是"不再搜那里"（如果已加载模块还依赖它，后续导入会连环失败）；以为改了 `sys.path` 之后已经 `import` 过的模块会受影响（不会，缓存优先）。

代码分析

```
>>> import sys
>>> sys.path.append('/Users/me/pycode4')    #
>>> import module                           #
```

解释器如何处理：`append` 直接就地修改 `sys.path` 这个列表对象（`sys.path.insert(0, dir)` 则插到最前，优先级最高）。下一次 `import module` 发起时，`PathFinder` 从左到右扫描这个已更新的列表。注意：修改必须发生在 `import module` 之前，`import` 完成后再加目录对已导入的模块没有影响；且修改只存在于当前进程。

设计思想：配置面与代码面在运行时可以互相转化——"路径"这种原本属于环境的东西，在 Python 里只是一段可以随手操纵的列表数据，这正是"可移植脚本"的底气所在。

22.14 Module File Selection（模块文件的选择）

英文原文

Besides folders, file types also factor into imports. As noted earlier, filename extensions (e.g., .py) are omitted from import statements by design: Python chooses the first file it can find on the search path that matches the imported name. This need not, however, be a .py source code file or a .pyc bytecode file, as the next section explains.

中文翻译

除了文件夹，文件的类型也参与导入的决策。如前所述，文件扩展名（如 .py）在 import 语句中被有意省略：Python 会挑选搜索路径上第一个与导入名字匹配的文件。不过，这个文件未必是 .py 源代码文件或 .pyc 字节码文件，正如下一节所解释的。

深度理解

- 核心概念：import 只认"名字"，不认"扩展名"——同一个名字可能对应多种实体，Python 取第一个命中的。
- 底层实现：PathFinder 对每个目录尝试一组后缀（.py、.pyc、.so/.pyd、目录等）与压缩格式检查，命中即返回 spec。
- 设计原因：省略扩展名让调用方与实现细节解耦——今天 b 是 b.py，明天换成 C 扩展 b.so，客户端代码一行不用改。
- 解决什么实际问题：第三方库常常把热路径用 C 重写后

替换同名模块，用户无感。

·初学者误区：以为 import 的就是"一个 .py 文件"（也可能是 .so、.pyc、目录、ZIP 成员.....）。

22.15 Module Sources (模块的来源)

英文原文

Really, imports are the point of interface to a host of external components—source code, bytecode, compiled extensions, ZIP files, Java classes, and more. Python automatically selects any type that matches a module's name. For example, an import statement of the form `import b` might today load or resolve to any of the following:

- A source code file named `b.py`
- A bytecode file in `pycache` named `b.cpython-312.pyc` or similar
- A bytecode file named `b.pyc` if no `b.py` source code file was located
- A directory named `b`, for package imports described in Chapter 24
- A compiled built-in module, coded in C and statically linked into Python when it is built
- A compiled extension module (e.g., `b.so` or `b.pyd`) coded in C and dynamically linked on import
- A source or bytecode file embedded in a ZIP file, automatically extracted when imported
- An in-memory image's module, for frozen (standalone) executables
- A Java class, in the Jython version of Python
- A .NET component, in the IronPython version of Python

Most of the items on this list extend `import b`

beyond simple files. To importers, though, differences in the loaded file type are completely irrelevant, both when importing and when fetching module attributes. Saying `import b` gets whatever module `b` is, according to your module search path, and `b.attr` fetches an item in the module, be it a Python variable or a linked-in C function. Some standard-library modules used in this book, for example, are actually coded in C, not Python; because they look just like Python-coded module files, their clients don't have to care.

中文翻译

事实上，导入是通往一大批外部组件（external components）的接口——源代码、字节码、编译扩展、ZIP 文件、Java 类等等。Python 会自动选择与模块名匹配的任何类型。例如，形如 `import b` 的导入语句，在今天可能加载或解析到下面任何一种：- 名为 `b.py` 的源代码文件- `pycache` 里名为 `b.cpython-312.pyc` 或类似的字节码文件- 如果没有找到 `b.py` 源代码文件，则可能是名为 `b.pyc` 的字节码文件- 名为 `b` 的目录，用于第 24 章描述的包导入（package import）- 编译好的内建模块，用 C 编写，在构建 Python 时静态链接进去- 编译好的扩展模块（如 `b.so` 或 `b.pyd`），用 C 编写，导入时动态链接- 嵌在 ZIP 文件里的源代码或字节码文件，导入时自动解出- 供冻结（独立）可执行文件使用的内存镜像模块- Java 类（在 Jython 版 Python 中）- .NET 组件（在 IronPython 版 Python 中）这份清单上的大多数条目都把导入扩展到了简单文件之外。但对导入方来说，被加载文件类型的差异完全无关紧要——无论是

导入时还是取模块属性时。说 `import b` 就得到根据你的模块搜索路径找到的那个模块 `b`，`b.attr` 就取模块里的一个成员——它可能是 Python 变量，也可能是链接进来的 C 函数。例如，本书用到的一些标准库模块其实是用 C 写的，不是 Python 写的；因为它们看起来和 Python 写的模块文件一模一样，它们的客户端根本不用关心。

深度理解

- 核心概念：`import` 是一个"多态"接口：同一句话 `import b` 背后可能是十种完全不同的实现来源。对调用方而言，`b.attr` 的语法永远不变。

- 底层实现：这正是 PEP 302 导入机制（finder 找到 loader）的威力——每种来源对应一个 loader（`SourceFileLoader`、`ExtensionFileLoader`、`zipimport`、`FrozenImporter` 等），它们都提供统一的 `loadmodule` 行为。

- 设计原因：统一接口 + 可插拔实现，让 Python 既能用 C 提速（`math`、`socket`），又能嵌入 ZIP、冻结进可执行文件，还不改任何用户代码。

- 解决什么实际问题：分发"独立可执行文件"（`PyInstaller` 等把模块冻进内存镜像）；把库打包成 ZIP 单文件分发；`Jython`/`IronPython` 生态的互通。

- 初学者误区：以为 Python 模块必须用 Python 写（标准库里一堆 C 模块）；以为看到 `b.so` 就是"病毒"或"加密"（它只是 C 写的扩展）；以为 ZIP 里的模块不能 `import`（`sys.path` 里有 `python312.zip`，`zipimport` 专门干这个）。

22.16 Selection Priorities (选择优先级)

英文原文

Given all the options listed in the prior section, there is a potential for conflicts. Python will always load the item with a matching name found in the first (leftmost) directory of your module search path, during the left-to-right scan of `sys.path`. But what happens if it finds multiple matching items in the same directory? In this case, Python follows a standard picking order, though this order is not guaranteed to stay the same over time or across implementations. In general, you should not depend on which type of file Python will choose within a given directory—make your module names distinct, or configure your module search path to make your module-selection preferences explicit.

中文翻译

鉴于上一节列出的种种选项，冲突的可能性是存在的。在从左到右扫描 `sys.path` 时，Python 总会加载在模块搜索路径第一个（最左）目录中找到的匹配项。但如果它在同一个目录里找到了多个匹配项呢？这种情况下，Python 会遵循一个标准的挑选顺序，不过这个顺序不保证随时间或跨实现保持不变。一般来说，你不应该依赖 Python 会在给定目录内选择哪种文件——把模块名字取得各不相同，或者配置模块搜索路径，让模块选择的偏好显式化。

深度理解

- 核心概念：两条优先级规则：①跨目录，取 `sys.path` 最左；②同目录内，有一个未公开保证的默认顺序。规则②不可依赖。
- 底层实现：目录扫描通常按 `SourceFileLoader` 与字节码优先、扩展次之的顺序（`FileFinder` 按已注册后缀的列表顺序尝试），但这是实现细节。
- 设计原因：不把"同目录同名不同后缀"的排序写死进规范，是为了给将来的文件类型、加载策略留空间。
- 解决什么实际问题：防止 `b.py` 和 `b.pyc` 并存时行为不确定——最佳实践是让名字唯一、用路径控制来源。
- 初学者误区：以为 `.py` 永远优先于 `.pyc`（不一定）；以为"目录里的文件顺序就是优先级"（不是，是注册的加载器顺序）；以为同目录同名不同扩展名是可以接受的设计（应避免）。

22.17 Path Outliers: Standalones and Packages（路径特例：独立可执行程序与包）

英文原文

Finally, while all of the foregoing reflects normal module usage, some tools bend the rules enough to merit a word in closing. For instance, module search paths are not relevant when you run standalone executables discussed in Chapter 2, which typically embed byt

ecode within their runnable files, and run without path settings. This is a delivery option outside the scope of this book, but see Appendix A for another overview of options in this domain. In addition, while it's syntactically illegal to include path and extension details in a standard import, package imports, covered in Chapter 24, allow import statements to include part of the directory path leading to a file as a set of period-separated names. Even so, package imports still rely on the normal module search path to locate the leftmost directory in a package path (they are relative to a directory in the search path), and cannot make use of any platform-specific path syntax in the import statements (such syntax works only on the search path). As hinted earlier, packages can also use "." syntax to restrict import searches—but we'll save this story for Chapter 24.

中文翻译

最后，尽管前面所述都是常规模块用法，但有些工具会把规则弯折到值得在结尾提一句的程度。例如，运行第 2 章讨论的独立可执行程序（standalone executable）时，模块搜索路径并不起作用——它们通常把字节码嵌入自己的可运行文件里，无需任何路径设置即可运行。这是一种超出本书范围的交付方式，但关于这个领域的选项概览可参见附录 A。此外，虽然在标准 import 中包含路径和扩展名细节在语法上是非法的，但第 24 章要讲的包导入（package import）允许 import 语句以点号分隔的名字集合形式，包含

通往文件的部分目录路径。即便如此，包导入仍然依赖常规模块搜索路径来定位包路径中的最左目录（它们是相对于搜索路径中某个目录的），并且不能在 `import` 语句中使用任何平台相关的路径语法（这类语法只作用于搜索路径）。正如前面暗示的，包还可以用 `"."` 语法来限制导入搜索——但这个故事我们留到第 24 章再讲。

深度理解

- 核心概念：搜索路径不是万能的——独立可执行文件完全绕开它；包导入则“部分”使用它（只定位最左目录）。

- 底层实现：独立可执行程序（PyInstaller/Py2exe 产物）把模块冻进镜像或归档，解释器从内存加载；包导入的 `import dir1.dir2.b` 先沿 `sys.path` 找到 `dir1`，再在 `dir1` 内部按点号逐层进入 `dir2`、`b`。

- 设计原因：两类特例分别服务两种交付形态——“发给没有 Python 的最终用户”（standalone）与“组织大型代码库”（package）。

- 解决什么实际问题：桌面应用分发不用装 Python；大型项目按目录组织模块，`import` 时只写相对名。

- 初学者误区：以为包路径的 `import` 可以写绝对路径或 `..`（不能，路径语法只能出现在搜索路径上）；以为 standalone 程序仍需配 `PYTHONPATH`（不需要，字节码已在镜像里）；以为 `"import a.b"` 中的点号与路径符号有映射关系（有，但那是第 24 章的故事）。

22.18 Chapter Summary (本章小结)

英文原文

In this chapter, we covered the basics of modules and explored the operation of import statements. We learned that imports find the designated file on the module search path, compile it to bytecode, and execute all of its statements to generate its contents. We also learned how to configure the search path to be able to import from directories other than the home and standard-library directories, primarily with PYTHONPATH settings. As this chapter also discussed, the import operation and modules are at the heart of program architecture in Python. Larger programs are divided into multiple files, which are linked together at runtime by imports. Imports in turn use the module search path to locate files, and modules define attributes for external use. The net effect divides a program's logic into reusable and self-contained software components. You'll see what this all means in terms of actual statements and code in the next chapter. Before we move on, though, let's run through the usual chapter quiz

中文翻译

在本章中，我们覆盖了模块的基础知识，并探讨了 import 语句的运作机制。我们认识到：导入先在模块搜索路径上寻

找指定的文件，把它编译成字节码，然后执行其中的所有语句以生成模块的内容。我们还学习了如何配置搜索路径——主要用 `PYTHONPATH` 设置——以便能够从主目录和标准库目录之外的其他目录导入。正如本章也讨论过的，导入操作和模块是 Python 程序架构的核心。大型程序被拆分为多个文件，在运行时通过导入链接成一个整体。导入反过来又用模块搜索路径定位文件，模块则为外部使用定义属性。净效果是把程序的逻辑划分为可复用、自包含的软件组件（components）。在下一章，你会看到这一切落实为具体的语句和代码意味着什么。在我们继续之前，还是先照老规矩做一遍本章的小测验吧。

22.19 Test Your Knowledge: Quiz (知识点测验)

英文原文

1. How does a module source code file become a module object? 2. Why might you have to set your `PYTHONPATH` environment variable? 3. Name the five major components of the module search path. 4. Name four file types that Python might load in response to an import operation. 5. What is a module namespace, and what does a module's namespace contain?

中文翻译

1. 模块的源代码文件是如何变成一个模块对象的？2. 你可能出于什么原因需要设置 `PYTHONPATH` 环境变量？3. 说

出模块搜索路径的五个主要组成部分。4. 说出 Python 针对一次导入操作可能加载的四种文件类型。5. 什么是模块命名空间，模块的命名空间中包含什么？

22.20 Test Your Knowledge: Answers (知识点测验：答案)

英文原文

1. A module's source code file automatically becomes a module object when that module is imported. Technically, the module's source code is run during the import, one statement at a time, and all the names assigned in the process become attributes of the generated module object. 2. You need to set `PYTHONPATH` only to import from directories other than Python's standard library and the "home" directory—the current directory when working interactively, or the directory containing your program's top-level file. In practice, this might be required for nontrivial programs that use libraries of tools. 3. The five major components of the module search path are the top-level script's home directory (the directory containing it), all directories listed in the `PYTHONPATH` environment variable, the standard-library directories, the site-packages root directory for third-party extension installs, and all directories listed in `.pth` path files located in standard places. Of these, programmers can customize `PYTHONPATH` and `.pth` files. 4. Python might load a source co

de (.py) file, a bytecode (.pyc) file, a C extension module, or a directory of the same name for package imports. Imports may also load more exotic things such as ZIP file components, Java classes under the Jython version of Python, .NET components under IronPython, and statically linked C modules that have no files present at all. In fact, with import extensions, imports can load nearly arbitrary items. 5. A module namespace is a self-contained package of variables, which are known as the attributes of the module object. A module's namespace contains all the names assigned by code at the top level of the module file (i.e., not nested in def or class statements). A module's global scope morphs into the module object's attributes namespace. A module's namespace may also be altered by assignments from other files that import it, though this is generally frowned upon (see Chapter 17 for more on the downsides of cross-file changes).

中文翻译

1. 模块的源代码文件在这个模块被导入时自动变成一个模块对象。从技术上讲，模块的源代码在导入期间被运行——一次运行一条语句——过程中被赋值的所有名字都会成为所生成模块对象的属性。2. 只有当你需要从 Python 标准库和"主目录"之外的其他目录导入时，才需要设置 PYTHONPATH。"主目录"指交互式工作时的当前目录，或包含程序顶层文件的目录。实践上，使用工具库的非平凡 (non trivial) 程序可能都需要它。3. 模块搜索路径的五个主要组

成部分分别是：顶层脚本的主目录（包含它的那个目录）、PYTHONPATH 环境变量中列出的所有目录、标准库目录、用于第三方扩展安装的 site-packages 根目录，以及位于标准位置的任何 .pth 路径文件中列出的所有目录。其中，程序员可以定制 PYTHONPATH 和 .pth 文件。4. Python 可能加载源代码（.py）文件、字节码（.pyc）文件、C 扩展模块，或是用于包导入的同名目录。导入还可能加载更奇特的东西，比如 ZIP 文件组件、Jython 版 Python 下的 Java 类、IronPython 下的 .NET 组件，以及磁盘上根本没有文件的静态链接 C 模块。事实上，借助导入扩展（import extensions），导入几乎可以加载任意条目。5. 模块命名空间是一个自包含的变量包，这些变量被称为模块对象的属性。模块的命名空间包含模块文件顶层代码（即不嵌套在 def 或 class 语句中的代码）所赋值的所有名字。模块的全局作用域会“变身”为模块对象的属性命名空间。模块的命名空间也可能被来自导入它的其他文件的赋值所修改，不过这种做法一般不太受欢迎（跨文件修改的缺点详见第 17 章）。

本章总结

技术拓展 (Technical Expansion)

实际项目中的应用场景

·项目骨架设计：入口脚本（main.py）只放控制流，业务逻辑按主题拆模块（如 database.py、utils.py、models /），全部放在同一个目录，零路径配置即可运行。

- CI/CD 与部署：CI 环境里用 PYTHONPATH 注入共享工具库目录；生产环境用 venv 隔离依赖；只读容器中加 -B 或预先生成 pycache 避免写缓存报错。
- 插件系统：运行时 sys.path.append + importlib.import_module 动态加载插件目录（很多框架就是这么干的）。
- 调试三板斧：python -c "import sys; print(sys.path)" 查看路径；python -X importtime -c "import os" 查看导入耗时；sys.modules.pop('x') 强制重新导入。
- 交付形态选择：源码目录交付、compileall -b 后的纯 .pyc 交付、或 PyInstaller 冻结成独立可执行文件——本章的三步流程与路径机制是理解这三种形态的基础。

与其他语言 (Java/C++) 的区别

维度	Python 模块	Java	C/C++
组织单元	.py 文件 / 包 (目录)	class / package	头文件 + 源文件
链接时机	运行时动态导入 (三步流程)	编译期类型检查 + 类加载器	预处理 #include + 链接器
命名空间	模块对象的属性字典	package 命名空间	全局符号 (易冲突)
缓存机制	sys.modules 字典 + .pyc	JVM 类缓存	链接器符号表
循环依赖	运行时解析, 基本可容忍	编译期报错	头文件包含顺序敏感
文件发现	sys.path 搜索 (扩展名省略)	classpath (需显式列出)	编译器 -I 路径

一句话总结：C 的 #include 是"编译期文本粘贴"，Java 是"编译期符号校验 + 运行期类加载"，而 Python 是"纯运行时三步流程 + 字典缓存"——这决定了 Python 的导入最灵活，也最容易被人误解。

Python 发展历史背景

- 早期 (1.x 时代) : 导入机制简单, .pyc 直接放在源码旁边, 文件名不带版本标签。
- Python 2.3 (2003) : PEP 302 引入 finder/loader 协议, 导入系统第一次可扩展 (import hooks) ——22.15 节"可加载任意类型"正是它的产物。
- Python 3.2 (2011) : 引入 pycache 子目录 + 文件名带实现/版本标签, 解决多版本 Python 的缓存冲突。
- Python 3.3 (2012) : PEP 420 引入隐式命名空间包; PEP 3147 (pyc 方案) 逐步定型。
- Python 3.7 (2018) : --check-hash-based-pycs 与哈希型字节码可用。
- Python 3.10 (2021) : 新增 sys.stdlibmoduleNames ; PEP 621 让打包元数据进入标准。
- Python 3.11 (2022) : 冻结模块 (frozen modules) 用于 os 等, 启动加速 10-15%。
- Python 3.12 (2023) : 更多冻结模块; importlib 性能优化; sys.monitoring 等新工具。
- 版本越低行为越"古早", 但三步导入流程与 sys.path 五大组件的骨架自 3.x 以来稳定不变。

高级开发者需要掌握的相关知识

- PEP 302 协议: findspec / modulefromspec / execmodule——写自定义 loader 的入口。

- `importlib.abc` 与 `MetaPathFinder`: 在 `sys.metapath` 里插队, 可以拦截并改写一切导入 (框架级技能)。
- `site` 模块与 `.pth` 执行语法: `.pth` 文件行首写 `import xx` 会在启动时执行, 是"启动钩子"的民间方案。
- ZIP 导入: `sys.path` 中的 `python312.zip` 与 `zipimport` 的实现原理。
- `venv` 内部机制: 它通过修改 `sys.prefix` / 生成 `pyvenv.cfg` 来"局部化"搜索路径——理解它需要本章的路径模型。
- `PYTHONSAFEPATH` / `-P`: 3.11+ 关闭"脚本目录进 `sys.path`"的安全选项 (避免意外导入同名脚本目录下的模块)。

学习建议 (Learning Advice)

·重要程度: ★★★★★ (5/5)。模块是 Python 程序组织的天花板, 本章是第五部分的地基, 不理解三步流程与搜索路径, 后续的 `from/reload` (23 章)、包与相对导入 (24 章)、`name/all` (25 章) 都无从谈起。

·应该掌握到什么程度:

1. 能脱口说出导入三步 (查找→编译→运行) 与 `sys.modules` 缓存的作用;
2. 能背出搜索路径五大组件, 并分别说出哪些自动、哪些可配置;
3. 会用 `PYTHONPATH`、`.pth`、`sys.path.append` 三种方式扩展路径, 并知道各自的生效范围 (单用户 / 单 Python)

n / 单进程) ;

4. 看到 pycache 里的 codefile.cpython-312.pyc 能立刻解读出全部信息;

5. 理解 .pyc 不是加密、不是语言的一部分, 只是缓存。

·后续应该学习的内容:

1. 第 23 章: import 与 from 语句的语法细节、reload 的真实语义、name 的用途;

2. 第 24 章: 包 (package) 与相对导入——本文件反复"预告"的内容;

3. 第 25 章: all、命名空间包、from 的隐藏风险;

4. 实践课: 用 venv + pip 安装第三方库, 观察它们落入 site-packages, 再回头看本章的路径模型, 会有"原来如此"的感觉。